



Friedrich-Alexander-Universität
Erlangen-Nürnberg

The Use of GenAI to Support the Formulation of Risk Control Measures

Master Thesis

Handed in by: Rithika Ravichandran

Matriculation number: 23177041

Study Program: Health and Medical Data Analytics (HMDA)

First supervisor: Prof. Dr. Jana Hutter

Second supervisor: M. Sc. Stephanie Heidepriem

‘ Editing time: 01.06.2025– 01.12.2025

Smart Imaging Lab, Radiological Institute
Henkestr. 91, 91054 Erlangen

Abstract

This thesis investigates how Generative Artificial Intelligence (GenAI) can be used to generate hazard keys for medical devices, particularly focusing on CT systems. Experts with extensive knowledge and experience in the field primarily carry out the industry's current manual workflow. And it's time-consuming. It needs careful coordination between teams that handle risk management and documentation.

In this context, two research question were established on which this thesis is focused on, (Research Question (RQ)01) how a GenAI-based chatbot could be utilized to generate hazard keys in a user-friendly manner; and (RQ02), how this system can be effectively implemented while ensuring data protection and compliance with regulatory requirements.

This project develops a prototype that integrates the Large Language Model (LLM) and Retrieval Augmented Generation (RAG) methods. The system uses a retrieval-first approach for the given function, hazard, and cause inputs. It uses semantic embeddings to search for the existing hazard key and determine the exact safety message for each search query. When no exact match is found for retrieval, the LLM is revoked to generate a new safety message with strict constraints. Semantic vectorization, an internal taxonomy, is used as a preprocessing pipeline to retrieve precision and reduce ambiguity in the generated hazard key.

The safety chatbot prototype at this stage has basic elements of governance by design built in; these include anonymized, purpose-limited data handling. Only the developer has access to the research dataset and validation logs to track all interactions with the chatbot. Human oversight is still very important; a domain expert will check all suggestions that were made, and they have the option to edit them if they want. As of now, the system is used only by the developer, so this practice is in place for the next iteration. The General Data Protection Regulation (GDPR) and the European AI law (EU AI ACT) set rules that are used as design guidelines to make sure that experiments are done in a responsible way without claiming full compliance.

A cognitive walkthrough was done with the experts in the field of the safety chatbot prototype, and a structured assessment of action sequences was conducted to see how usable it is. The results show that the system can reliably find the accurate safety messages and make sense of the suggestions when there isn't an exact match. Some limitations still exist, such as being sensitive to inputs that aren't in the taxonomy, having hallucinations from time to time, and relying on the completeness of the underlying dataset.

The thesis shows that GenAI can aid with the hazard documentation process in a suggestive way when it's based on a retrieval and overseen by experts. Future work should focus on enterprise integration, role-based access control, automated validation pipelines, and adding more data sets so that a new hazard key can be generated that is not a part of the current taxonomy.

Contents

Abstract	i
Contents	iii
List of Figures	vi
Listings	viii
Appendices	viii
List of Abbreviations	ix
1 Introduction	1
1.1 Motivation	4
1.2 Problem Statements and Research Questions	6
1.3 Research Method	7
1.4 Research Goals and Scientific Approach	8
1.5 Structure of Thesis	10
2 State of the Art in Science and Technology	12
2.1 Generative Artificial Intelligence	13
2.1.1 Natural Language Processing and Large Language Models	15
2.1.2 Retrieval Augmented Generation	20
2.1.3 Recursive Abstractive Processing for Tree-Organized Retrieval	22
2.1.4 Discussion and Intermediate Summary	24
2.2 Risk Control Workflow – Hazard Keys	25
2.2.1 Definition of Key Terms	25
2.2.2 Risk Communication in User Documentation	30
2.2.3 Discussion and Intermediate Summary	31
2.3 Data Protection and Regulatory Requirements for AI Systems	32
2.3.1 General Data Protection Regulation (GDPR)	33
2.3.2 EU Artificial Intelligence Act (AI Act)	34
2.3.3 Trustworthy AI	35
2.3.4 Discussion and Intermediate Summary	36

2.4	AI Governance Frameworks and Compliance Strategies	37
2.4.1	Hourglass Model	37
2.4.2	Discussion and Intermediate Summary	41
2.5	Complete Summary and Discussion	41
3	Design and Concept Modeling	43
3.1	Design Method	43
3.2	Context of Use	45
3.3	Use Case Model	48
3.3.1	Requirements	48
3.3.2	Use Case Diagram	49
3.3.3	Description of the Use Cases	50
3.4	Design Solutions per Use Case	51
3.4.1	Information model	51
3.4.2	Sequential Model	52
3.4.3	Component Model	54
3.4.4	User Interface Model	57
3.4.5	Architecture Model	58
3.5	Summary and Discussion	59
4	Implementation	60
4.1	Core Technologies	60
4.2	Safety chatbot	61
4.2.1	Implementation Setup and Data Preparation	61
4.2.2	Retrieval-Augmented Generation Pipeline	64
4.3	Compliance Architecture	66
4.4	Discussion and Summary	70
5	Evaluation	72
5.1	Cognitive Walkthrough	72
5.2	Evaluation Setup and Phases	73
5.2.1	Input Parameters	73
5.2.2	Action Sequences	74
5.2.3	Examination of the Sequence of Actions	74
5.3	Results and Analysis	76
5.3.1	Logging Critical Information	76
5.4	Discussion and Summary	77

6	Conclusion	80
6.1	Addressing the Research Questions	80
6.2	Future Research and Work	82
	Appendices	84
A	Appendix	84
A.1	User Interface Views	84
A.2	Cognitive Walkthrough	85
	Cognitive Walkthrough	85
A.2.1	Application Initialization and System Launch	85
A.2.2	Safety Message Generation Workflow	87
A.2.3	Message Validation and Hazard Key Creation	89
A.2.4	Administrative Oversight and Data Governance	91
A.3	User Documentation	92
A.3.1	Activate the Python Virtual Environment	92
A.3.2	Build the Index and Embeddings	93
A.3.3	Launch the Streamlit Application	93
A.3.4	Enter Input Data	93
A.3.5	Retrieve or Generate Safety Message	94
A.3.6	Review, Edit, and Confirm	94
A.3.7	Export the Results	94
A.3.8	Completion and Administration	94
	Bibliography	95
	Declaration of authorship	102
	Acknowledgements	103

List of Figures

Figure 1.1:	Example of a safety message labeled Warning shown to alert users of a serious risk.	2
Figure 1.2:	Example of a safety message labeled Caution shown to alert users of a moderate risk.	2
Figure 1.3:	Example of Safety Message	3
Figure 1.4:	Framework according to Nunamaker/Chen[43]	8
Figure 2.1:	Illustration of tokenization and word embeddings	16
Figure 2.2:	Simplified Transformer architecture	18
Figure 2.3:	Overview of the Retrieval-Augmented Generation architecture. .	21
Figure 2.4:	Flowchart to represent the terms	29
Figure 3.1:	User-Centric System Design (UCSD) framework adapted according to Mayhew [38]	44
Figure 3.2:	Adapted UCSD workflow applied in this thesis	45
Figure 3.3:	Manual Workflow for Hazard key creation	46
Figure 3.4:	Chatbot workflow	47
Figure 3.5:	Context Diagram	48
Figure 3.6:	Use Case Diagram	50
Figure 3.7:	Information model	52
Figure 3.8:	Sequential model	54
Figure 3.9:	Components model	56
Figure 3.10:	The User Interface model	57
Figure 3.11:	Conceptual architecture model	58
Figure 4.1:	Input dataset used for embedding and retrieval.	62
Figure 4.2:	User interface displaying a retrieved safety.	65
Figure 4.3:	Overview of the compliance architecture.	70
Figure A.1:	Interface displaying generated safety message options when no exact match exists.	84

Figure A.2:	Manual editing and confirmation interface for refining generated safety messages before final export.	85
Figure A.3:	Activation of the virtual environment.	85
Figure A.4:	Execution of the indexing script (build_index.py).	86
Figure A.5:	Embedding model initialization and vector dimension output. . .	86
Figure A.6:	Streamlit application launched successfully.	86
Figure A.7:	User interface loaded and system ready for interaction.	86
Figure A.8:	Structured Input Entry.	87
Figure A.9:	Retrieval Initialization/Database Query Execution.	87
Figure A.10:	Exact Reference Retrieval	87
Figure A.11:	No exact match, the LLM generates a message.	88
Figure A.12:	Display of Retrieved Messages/ Review of Message Options. . . .	89
Figure A.13:	Message Selection and Optional Editing.	90
Figure A.14:	Confirmation of Selection /Hazard Key Assignment	90
Figure A.15:	Final Message Confirmation and Export Readiness.	91
Figure A.16:	Export Trigger.	91
Figure A.17:	Dataset Update.	91
Figure A.18:	Validation Log Append.	92
Figure A.19:	File-Lock Handling.	92
Figure A.20:	Administrative Review and Archival.	92

Listings

Listing 1: Code snippet showing data import and validation checks.	62
Listing 2: FAISS index creation and local storage process.	63
Listing 3: Core retrieval and generation logic.	64
Listing 4: Excerpt of the validation log routine ensuring traceability.	68

List of Abbreviations

CT	Computed Tomography
AI	Artificial Intelligence
NLG	Natural Language Generation
LLM	Large Language Model
GPT	Generative Pre-trained Transformer
GDPR	General Data Protection Regulation
PS	Problem Statement
RQ	Research Question
HIPAA	Health Insurance Portability and Accountability Act
RG	Research Goal
GenAI	Generative Artificial Intelligence
GAN	Generative Adversarial Network
VAE	Variational Autoencoder
RAG	Retrieval Augmented Generation
NLP	Natural Language Processing
RAPTOR	Recursive Abstractive Processing for Tree-Organized Retrieval
NER	Named Entity Recognition
RNN	Recurrent Neural Network
FFN	Feed-Forward Networks
BERT	Bidirectional Encoder Representations from Transformers
RAG	Retrieval Augmented Generation
FAISS	Facebook AI Similarity Search
RCM	Risk Control Measures
COSIMA	Component Oriented Structured Information Management Application
IFU	Instructions for Use
GUI	Graphical User Interfaces
EU AI ACT	European AI law
DPIA	Data Protection Impact Assessments
EHDS	European Health Data Space

RBAC	Role Based Access Control
MFA	Multi-Factor Authentication
MRI	Magnetic Resonance Imaging
UCSD	User-Centric System Design
EU	European Union
HLEG AI	High-Level Expert Group on Artificial Intelligence
UI	User Interface
CPU	Central Processing Unit
ISO	International Organization for Standardization
UC	Use Case
SSO	Single Sign-On
PLM	Product Lifecycle Management
QMS	Quality Management System
RMS	Risk Management System
HRK	Hazard Requirement Key
FHC	Function–Hazard–Cause
CW	Cognitive Walkthrough
AS	Action Sequences
API	Application Programming Interface
JSON	JavaScript Object Notation
MVC	Model–View–Controller
NTFS	New Technology File System
AS	Action Sequences
OQ	Open Question
HRK	Hazard Requirement Key

1 Introduction

Computed Tomography (CT) has been a key part of medical imaging since Sir Godfrey Hounsfield and Dr. Allan Cormack came up with it in the early 1970s, [21]. Over the years, CT technology has gone from simple imaging systems to very advanced diagnostic tools [55].

Big medical imaging companies have pushed for constant innovation by coming up with new technologies like spiral CT, multi-slice CT, and quantum counting CT. These technological advances have made scans faster, lowered radiation exposure, and greatly improved the quality of the images. Recent technological innovations have greatly improved diagnostic precision and elevated the quality of patient care around the world.

To ensure that CT systems are used safely and effectively, you need to have a structured way to manage risk and quality. This principle remains crucial, regardless of the evolution of technology. One important part of this philosophy is having a thorough risk management process in place [6].

This system is intended to ensure that risks are identified, labeled, tracked, documented, and mitigated [46]. CT systems, like all medical devices, must meet strict regulatory standards [24], which call for clear and thorough risk management. Such documentation includes complete records of all risk-related information, such as risk control measures, safety messages, hazards, and hazard keys.

A well-organized framework for making hazard keys is an important part of risk documentation. These keys link each identified hazard to the appropriate system function, its potential causes, and the actions that can mitigate the risk. Monitoring these key elements allows risks to be detected and mitigated at an early stage. Such an approach makes it safer and more effective to use CT systems in real-life clinical workflows.

A hazard becomes a risk when assessed in terms of its severity and the probability of occurrence [19]. Not all identified hazards require mitigation; only those exceeding acceptable risk thresholds as defined by regulatory standards do. Crucially, risks do not arise solely from technical failures. User errors and operator misuse are equally important and must be addressed during risk analysis. [61].

Mitigation strategies often involve safety messages, classified as either warnings, which address more serious potential consequences as seen in Figure 1.1, or cautions to prevent minor harm as seen in Figure 1.2 [24]. The choice depends directly on the severity and likelihood of the associated harm. This structured and systematic process is essential to fulfill regulatory expectations (*e.g.*, International Organization for Standardization (ISO) 14971) and to ensure patient safety across all clinical use scenarios. Figure 1.1 and Figure 1.2 have been sourced from the Risk Management Plan [64].

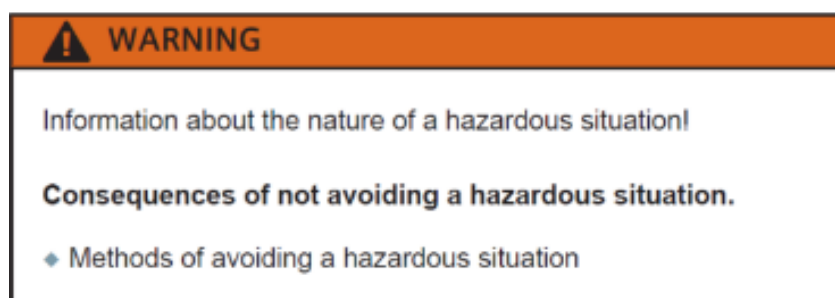


Figure 1.1: Example of a safety message labeled Warning shown to alert users of a serious risk.

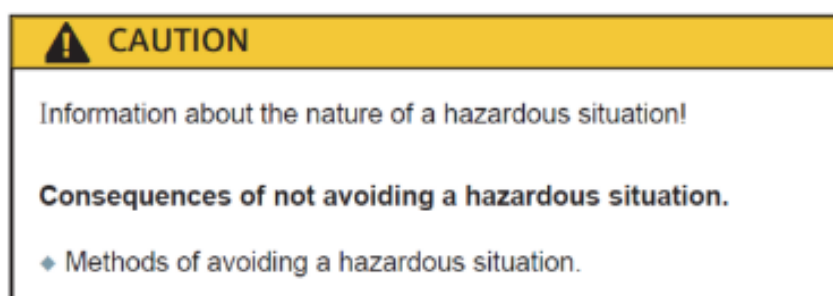


Figure 1.2: Example of a safety message labeled Caution shown to alert users of a moderate risk.

The development of hazard keys is a collaborative process. It involves expert discussions, review of communication records (*e.g.*, email chains), analysis of customer complaints and feedback, and evaluation of issues reported by medical professionals.

When a risk cannot be sufficiently reduced through design modifications such as software or hardware adjustments, a corresponding safety message is incorporated into the documentation to inform and guide users.

We formulate the safety message based on a specific function, hazard, and cause scenario. In this case, the function involves registering patients on the patient orientation form. The identified cause is the incorrect entry of patient orientation, which may lead to an incorrect diagnostic basis. The resulting safety message is a caution instructing the user to verify patient positioning.

Each safety message consists of 4 main parts [64]:

- Signal word – Specifies whether the measure is classified as danger, warning, caution, or notice
- Cause – What is a hazard? (*e.g.*, electric shock, crushing, radiation)
- Consequence - Describes the potential impact of the hazard if the failure occurs.
- Countermeasure – Defines the specific action, modification, or guideline to mitigate the identified risk.

This structured format, as seen in Figure 1.3, ensures that the user is clearly informed about the source of the risk, the potential consequences, and the required mitigation measures. Such messages are crucial for maintaining patient safety and fulfilling regulatory requirements [22, 64].

Operator Manual: CAUTION
Source: Wrong entry of patient position!
Consequence: Wrong basis for diagnosis.
Countermeasure:
Make sure that the patient position is correct.

Figure 1.3: Example of Safety Message

Artificial Intelligence (AI) is now widely integrated across medical imaging workflows, assisting in tasks such as patient positioning, protocol optimization, and image reconstruction in CT systems. While AI enhances diagnostic efficiency and precision, it serves to complement rather than replace human radiological expertise.

The field of Natural Language Generation (NLG) has undergone remarkable growth, particularly with the rise of LLMs [5]. NLG refers to the process by which AI systems generate human-like text from structured or unstructured data [17]. These systems use sophisticated algorithms to analyze input information, recognize patterns, and construct coherent, contextually appropriate responses.

A breakthrough in this area was the release of ChatGPT, developed by OpenAI. Based on the Generative Pre-trained Transformer (GPT) architecture, it demonstrated the ability to generate structured, fluent, and legible text with minimal human supervision [44]. The boom of LLMs following ChatGPT's success has revolutionized industries, enabling automation in report writing, text formatting, note-taking, and email drafting.

LLMs can understand context, adapt to different writing styles, and produce highly fluent outputs, making them an essential tool for a wide range of applications. Given their ability to process large amounts of data, LLMs presents a promising solution to automate and optimize the creation of safety messages and their corresponding hazard key generation.

By integrating an AI-powered chatbot, it is possible to extract critical information from existing hazard records, analyze risk factors, and generate standardized, accurate, and structured safety messages in real-time. This approach can improve efficiency, consistency, and compliance while reducing the workload of human experts. The proposed system will be an AI-powered chatbot designed to assist in the creation of safety messages and Hazard Requirement Key (HRK) generation for CT systems. The chatbot will accept function, hazard, and cause as input and generate a corresponding safety message and hazard key as output.

1.1 Motivation

Innovation in medical imaging has improved drastically, especially in CT technology, which has transformed the diagnostic abilities and the patient outcomes. At the same time, maintaining the system for safety and keeping up with the regulatory requirements does get hard. Right now, the whole risk documentation workflow is still mostly manual, time-consuming, and prone to inconsistencies, thus making it difficult to maintain transparency and traceability. This highlights the need for

intelligent, structured, and compliant solutions that can support safety engineers in identifying hazards, managing risks, and maintaining regulatory standards through traceable documentation.

Currently, the risk documentation workflow is a manual, labor-intensive process involving the detailed analysis of system functions, hazard identification, root cause investigation, and the formulation of corresponding safety messages or mitigation strategies. Experts must ensure that all documented risks comply with stringent regulatory frameworks such as ISO 14971 [24] and GDPR [63]. While thorough, this traditional method is time-consuming, prone to human oversight, and struggles to scale alongside the rapid pace of technological innovation.

This is where LLMs offers a compelling solution [28]. LLMs are capable of parsing extensive medical documentation, extracting patterns, identifying risk relationships, and generating consistent, accurate, and structured content such as safety messages and hazard keys [41]. By leveraging an AI powered chatbot, the manual workload can be significantly reduced [34]. Such a system can assist experts by generating first draft suggestions for safety message and hazard key list, based on inputs like function, hazard, and cause [56]. This not only increases the speed of the documentation but also improves the quality, consistency, and compliance of the outputs [26].

The proposed chatbot preserves human oversight, making sure that all critical safety decisions remain in the hands of qualified professionals, while the AI handles the repetitive and administrative workload. In this way, the system contributes to a more robust and adaptable risk management process that reduces cognitive load, minimizes burnout, and improves overall productivity and job satisfaction for the teams responsible.

The involvement of AI systems gives rise to new risks. The underlying AI model can be continuously updated to incorporate new regulatory requirements and clinical insights. This ensures that the risk documentation process remains current, scalable, and aligned with the latest safety standards.

On a personal level, this research puts together my academic background in medical engineering, hands-on experience within CT development workflows, and along with my interest in applying AI to practical healthcare challenges. It represents an opportunity to contribute to a domain that is working on improving the efficiency, consistency, and transparency in safety documentation. It, in turn, increases patient

safety and regulatory compliance by strengthening the overall reliability. Building on this motivation, the next section outlines the core problems addressed in this study and formulates the research questions that guide its direction.

1.2 Problem Statements and Research Questions

In automating the creation of hazard keys for CT systems, several critical challenges must be addressed to ensure that an AI-powered solution operates effectively, accurately, and in compliance with both internal quality frameworks and external regulatory requirements. Across the medical imaging industry, this process remains largely manual, requiring substantial expert input and coordination between safety, engineering, and regulatory teams. To enable a reliable and scalable AI-driven approach, this research focuses on identifying and addressing these key challenges within the broader context of medical device risk management. To address this issue, the following Problem Statements (PSs) and Research Goals (RGs) were formulated. The first Problem Statement (PS) along with its Research Goal (RG) is as follows:

PS1: It is not clear how GenAI systems can be leveraged to build a user-friendly chatbot for generating hazard keys

A key challenge in automating the creation of the hazard key is ensuring that the chatbot accurately maps input parameters, functions, hazards, and causes to the corresponding safety message and Hazard Requirement Key (HRK). Due to the interconnected and context-sensitive nature of these parameters, even slight variations in input can lead to different yet equally valid outputs. Traditional rule-based systems struggle with these complexities, highlighting the need for AI-driven techniques that enable context-aware retrieval and the intelligent generation of hazard keys.

RQ1: *How can a GenAI system be designed to build a user-friendly chatbot for hazard key generation?*

Now we can move to the second Problem Statement (PS) and its corresponding Research Goal (RG)

PS2. It is not clear how GenAI can be used while ensuring data protection and regulatory compliance.

The healthcare industry, including organizations like Siemens Healthineers, operates under strict regulatory frameworks such as the GDPR [63] and Health Insurance Portability and Accountability Act (HIPAA) [60], which mandate stringent standards for data privacy and security. Implementing an AI-driven system for hazard key list generation introduces specific challenges, particularly when leveraging open-source LLMs that may require internet access or rely on external servers. To ensure safe and compliant deployment within healthcare environments, it is essential to maintain a controlled infrastructure that safeguards sensitive data while adhering to all applicable regulatory requirements. This problem leads to RQ2.

RQ2: *How can GenAI be effectively used while ensuring data protection and compliance with regulatory requirements?*

1.3 Research Method

This research follows the framework proposed by Nunamaker et al. [43], which defines four interconnected phases: observation, theory building, system development, and experimentation, as seen in Figure 1.4. These phases provide a structured yet adaptable approach to address the challenges of automating hazard key list generation for CT systems.

We analyze the current advancements on the subject throughout the observation phase. The aim is to identify which existing methods and technologies could contribute to solving the task. Additionally, it is necessary to determine where current gaps exist that must be addressed over the course of the work.

Using insights from the observation phase, the theory-building phase develops conceptual models and theoretical foundations. This phase focuses on defining the core principles, frameworks, and assumptions that guide the solution approach.

During the system development phase, the proposed solution is put into action based on the theoretical framework. It covers the system’s architectural design and technical implementation, ensuring that functional and non-functional criteria are met.

In the experimentation phase, the developed system is tested and evaluated. The goal is to assess its usability through defined action sequences and gather feedback. The results help validate the approach and inform necessary improvements.

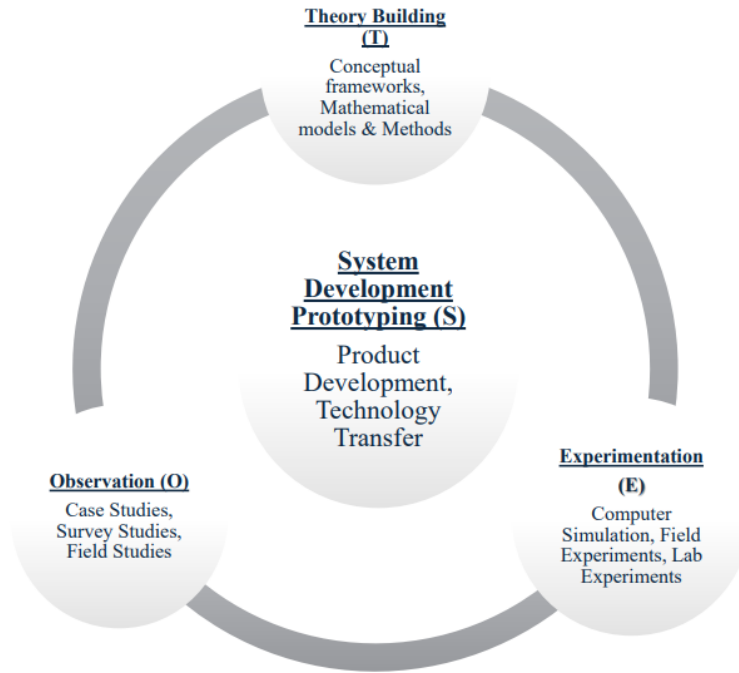


Figure 1.4: Framework according to Nunamaker/Chen[43]

This systematic and iterative research design ensures a robust investigation that is both theoretically grounded and practically relevant.

1.4 Research Goals and Scientific Approach

This thesis aims to design a GenAI system that will create a user-friendly chatbot capable of supporting hazard key generation. The system should be capable of processing the given inputs and accurately suggesting the corresponding hazard keys clearly and consistently.

To address RQ1, several RGs have been defined. The observation phase begins with understanding how GenAI systems work and how such systems can identify

safety messages from inputs such as function, hazard, and cause (RGO1.1). This involves analyzing existing systems and documentation practices to determine how safety-relevant information is retrieved and utilized. Additionally, it is important to understand how hazard keys are currently created, what parameters are considered, and who the end users are (RGO1.2).

In the theory-building phase, a conceptual framework is developed to guide the system's logic and decision-making process (RGT1.1). This framework defines how the system learns and maps input data to the appropriate hazard key. Using the insights from both the observation and theory phases, the system development phase involves building a prototype of the chatbot (RGS1.1). The chatbot is designed to process user input and retrieve or generate the relevant hazard key accurately while maintaining compliance with medical documentation standards. Finally, in the experimentation phase, the chatbot is evaluated (RGE1.1).

The remaining research goals focus on RQ2. The first step is to analyze the relevant data protection and regulatory requirements specific to AI systems (RGO2.1). In the subsequent phase, it is imperative to identify the existing AI governance frameworks and compliance strategies used in similar contexts (RGO2.2) to gain insights on best practices and necessary actions. A compliance framework that combines privacy-preserving methods and AI governance strategies will be modeled (RGT2.1) and developed (RGS2.1) based on the results of the observation goals. In the final phase, the effectiveness of maintaining compliance, focusing on its ability to protect sensitive data while accurately assigning hazard keys, will be evaluated (RGE2.1). Prioritizing these *RGs* will guide the systematic exploration and development throughout this thesis.

In chapter 2, a detailed literature review will be conducted focusing on the following RG related to observation activities:

- RGO1.1: Understand the use of GenAI for hazard key generation.
- RGO1.2: Analyze hazard key generating workflow.
- RGO2.1: Data protection and regulatory requirements for AI systems.
- RGO2.2: Identify existing AI governance frameworks and compliance strategies.

In chapter 3, the conceptual frameworks and theoretical foundations are developed to support the design and compliance of the system:

- RGT1.1: Develop a conceptual framework for mapping inputs to safety messages and hazard keys.
- RGT2.1: Create a compliance framework integrating privacy preserving techniques and AI governance principles for safe, regulatory compliant operation.

Chapter 4 focuses on the development of the GenAI chatbot system:

- RGS1.1: Design and build a chatbot that accurately processes inputs and generates corresponding safety messages and hazard keys.
- RGS2.1: Incorporate data protection mechanisms and regulatory compliance features within the system architecture.

And in chapter 5, the system prototype is evaluated for usability

- RGE1.1: Assess whether the chatbot retrieves existing safety messages correctly and generates coherent suggestions when required, based on structured action sequences and expert review..
- RGE2.1: Evaluate how the prototype embeds basic data protection and governance measures and whether these design choices align with key principles from relevant regulations.

1.5 Structure of Thesis

This thesis is divided into six chapters, each addressing key components of the research.

Chapter 2 - implements the Observation phase and is the State of the Art in Science and Technology. This chapter explores the foundational literature and current advancements in AI, LLMs, and risk management workflows, including the process of hazard key generation. It also examines data protection principles and relevant regulatory requirements for AI systems, providing the theoretical and technical basis for the development of this thesis.

Chapter 3 - the theory-building phase, focuses on the design and concept modeling. In this chapter, a user-centered system design approach is employed to ensure that the AI solution aligns with user needs and practical requirements. It describes how user requirements are collected and integrated into the conceptual modeling and design process, focusing on usability, functionality, and relevance to risk management workflows and hazard key generation.

Chapter 4 - the system development phase, shows how the system is implemented. This chapter details the development of the system prototype based on the concepts and design principles established in the previous chapter. The prototype implementation serves as a practical realization of the design.

Chapter 5 - is about Experimentation and Evaluation. In this chapter, we evaluate the prototype's usability through a proposed cognitive walkthrough.

Chapter 6 - the final chapter summarizes the main findings, discusses the implications and limitations of the research, and proposes future directions for improving and extending the AI system.

2 State of the Art in Science and Technology

This chapter presents the scientific and technical background relevant to developing a GenAI system for hazard key generation. The purpose is to provide an overview of current technologies, regulatory considerations, and governance strategies that inform the modeling and implementation phases of this work.

Section 2.1 introduces the concept of GenAI by addressing RGO1.1, outlining its definition, main types, and relevance to this study. The section focuses specifically on the use of LLMs for handling text-based tasks. It provides a brief overview of how LLMs function and highlights its potential to support structured documentation processes in highly regulated environments, such as those found in medical device development and risk management.

Section 2.2 examines the workflows and structure involved in generating hazard keys by addressing RGO1.2, particularly in the context of medical devices such as CT systems. Understanding the documentation process and its challenges is essential to identifying how AI can provide meaningful support.

Section 2.3 addresses RGO2.1, where data protection and legal requirements must be considered when implementing AI in safety-critical domains. This includes key principles of data privacy, which are especially relevant in the healthcare and medical technology sectors.

Section 2.4 reviews existing AI governance frameworks and compliance strategies by addressing RGO2.2. These guides ensure transparency, accountability, and alignment with regulatory expectations when using GenAI.

Finally, Section 2.5 presents a focused discussion along with an intermediate summary of the key insights. These findings will guide the theory building and framework development in Chapter 3.

2.1 Generative Artificial Intelligence

Advances in AI have contributed to the development of systems that are capable of analyzing complex information and producing new, meaningful outputs. This emerging area is often referred to as GenAI, and it is beginning to reshape how digital content is created, automated, and adapted across a wide range of fields, including design, engineering, medicine, and finance [3].

At its core, GenAI refers to a class of machine learning techniques that enable systems to learn the underlying distribution of data and generate new instances that resemble the original distribution [18, 32]. These models are typically trained on large-scale, multi-modal datasets and are capable of generating diverse outputs, including text, images, audio, code, and structured data, depending on the application [3]. The primary types of technologies used in this context include Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and, more recently, LLMs.

GANs is a type of deep learning model used to generate new data that is similar to the data it was trained on. A GAN consists of two neural networks trained in opposition: the generator creates data samples, while the discriminator checks whether these samples are real or produced by the model. This adversarial interaction drives both networks to improve, ultimately enabling the generation of highly realistic synthetic data [18]. GANs have demonstrated practical success in industrial design and prototyping, for instance, in automotive and appliance development, where generative models accelerate concept exploration and variant creation [3].

VAEs is another major class of generative models. Unlike GANs, which generates data through adversarial training, VAEs uses a probabilistic modelling approach that compresses data into a latent representation and then reconstructs it through a decoder. During training, the model optimizes both reconstruction accuracy and the ability to sample from a structured latent distribution, typically Gaussian [32].

In the context of medical imaging, VAEs have become valuable for anomaly detection and unsupervised learning of normal anatomical patterns. By learning compact representations of healthy anatomical structures, a trained VAE can identify deviations when presented with abnormal inputs. For example, in brain Magnetic Resonance Imaging (MRI) scans, VAEs trained on healthy datasets can reconstruct normal-like images even when the input contains pathologies, enabling the localization of abnormal regions through reconstruction error [2].

More recently, LLMs, such as OpenAI’s GPT-4, have demonstrated exceptional performance in language centric tasks due to their ability to model complex linguistic relationships and reason over domain specific content [5, 41]. Their capacity to process both structured and unstructured text makes them highly suitable for regulated documentation workflows. LLMs can assist in identifying and formulating safety messages by interpreting structured inputs and generating coherent, traceable outputs aligned with safety and regulatory standards [59].

This section, as defined in RGO1.1, aims to investigate how GenAI systems can be applied to hazard key generation, with a focus on identifying safety messages from structured data inputs. Achieving this objective requires a technical understanding of foundational GenAI components, particularly LLMs and RAG, alongside a critical analysis of existing documentation practices, stakeholder needs, and workflow constraints.

To support this investigation, the following subsections are organized to first introduce the key concepts of Natural Language Processing (NLP) and LLMs. Subsection 2.1.1 provides the theoretical basis for language-centric AI models. Subsection 2.1.2 explores the RAG framework, which enhances LLMs through external knowledge retrieval to improve factual grounding [35]. Subsection 2.1.3 examines Recursive Abstractive Processing for Tree-Organized Retrieval (RAPTOR), a hierarchical retrieval and generation architecture particularly suitable for structured documentation workflows [53]. Finally, Subsection 2.1.4 provides an intermediate discussion and summary of how GenAI can be effectively integrated into hazard key generation processes.

Together, these subsections establish a comprehensive technological foundation for evaluating the applicability of GenAI in safety-relevant documentation workflows, forming the basis for designing systems that are intelligent, context-aware, and

compliant with the operational and regulatory requirements of risk management in healthcare.

2.1.1 Natural Language Processing and Large Language Models

NLP is the foundation that GenAI systems use to understand, interpret, and produce human language in a meaningful way [30]. It draws from linguistics, computer science, and AI, allowing machines to work with natural text and speech as humans do [67]. At its core, NLP makes it possible for computers to analyze language data, extract relevant information, and generate coherent text outputs [30]. NLP systems perform a diverse range of tasks that enable machines to process and understand human language effectively [67].

These capabilities include syntactic parsing, which analyzes the grammatical structure of sentences; semantic understanding, which interprets the meaning of words within context; and Named Entity Recognition (NER), which identifies and categorizes key information such as medical terms, dates, or device names[30]. Furthermore, advanced tasks such as machine translation and automatic summarization have been significantly improved, allowing cross-lingual communication and efficient information compression while preserving essential meaning [30, 67].

Risk management documents typically follow standardized formats, use formal language, and contain logical structures. They often include repetitive patterns and terminology that need to be consistently interpreted and maintained. NLP can support this process by recognizing recurring elements, tracking relationships between different parts of the text, and extracting structured data from unstructured inputs [10, 67].

NLP has evolved significantly from early rule-based and statistical methods to deep learning based approaches that utilize large datasets and neural networks [30, 67]. A fundamental preprocessing step is tokenization, where input text is broken down into smaller units called tokens. Depending on the tokenization approach, a token may represent an entire word, a subword unit, or even a single character [54]. For instance, in the sentence “The cat chases the mouse”, each word is treated as an individual token. These tokens are then mapped to word embeddings, which are

numerical representations that capture semantic meaning in a format the model can process [30].

As visualized in Figure 2.1, each word from the sentence is converted into a dense vector in an embedding space. These embeddings preserve relationships between words, for example, capturing that “cat” and “mouse” are semantically related within a context. Tokenization and embeddings are essential for enabling NLP models to flexibly interpret diverse and domain-specific language, including rare or specialized terms [30, 54].

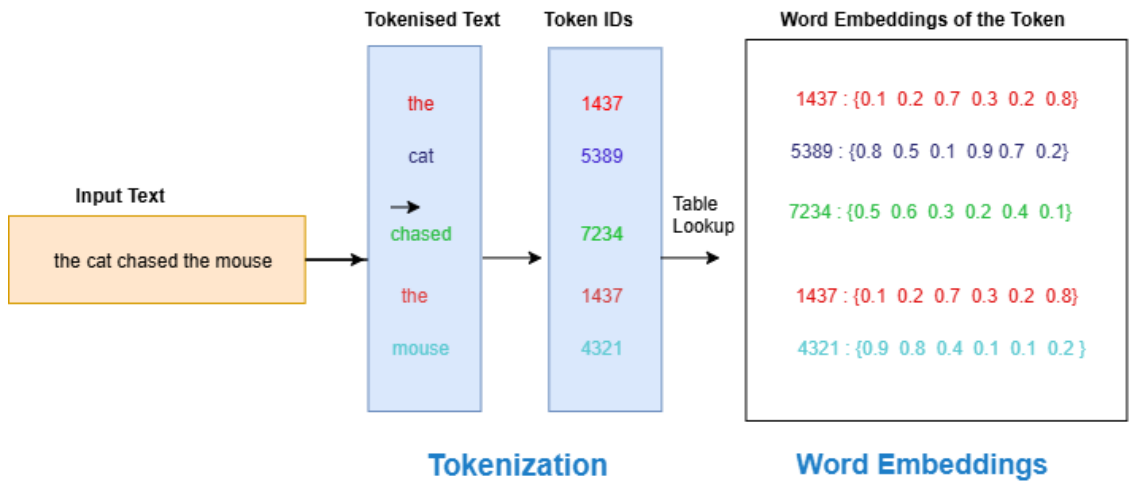


Figure 2.1: Illustration of tokenization and word embeddings

The success of LLMs is largely driven by the transformer architecture, introduced by Vaswani et al. [62], which significantly changed the way language models are made and trained. Unlike earlier sequence models such as Recurrent Neural Networks (RNNs), which process input tokens one at a time and often struggle with long-range dependencies, transformers operate on entire sequences simultaneously [52].

This parallelism is enabled by the self-attention mechanism, which allows the model to assess the importance of each word relative to every other word in the sequence [52, 62]. For example, in a sentence describing a device malfunction, the model can effectively identify which terms refer to the cause, the hazard, and the resulting harm, even if these words are far apart in the sentence [9].

The Transformer architecture, as illustrated in Figure 2.2, is composed of two primary components: the encoder and the decoder, each structured as a stack of layers that include multi-head self-attention, Feed-Forward Networks (FFN), and layer normalization [52, 62].

The process begins with the input sentence, which is first transformed into word embeddings, dense numerical vectors that capture both the semantic meaning of each word and its context [30, 52]. The encoder then processes these embeddings using multi-head self-attention, allowing the model to analyze all token-to-token relationships in parallel [62]. This enables the model to understand, for example, how “overheating” and “device failure” relate to each other in a technical report [59]. The encoder outputs a set of contextualized embeddings that reflect this understanding. The decoder takes as input both the encoder’s representations and the partial sequence of tokens it has already produced. It applies masked self-attention, which means the model is only allowed to look at previous tokens and not future ones. A masking layer blocks all tokens to the right, ensuring that the decoder predicts each next word step by step without “seeing ahead”. In addition to this, the decoder uses cross-attention to integrate the contextual information coming from the encoder’s output. While masked self-attention focuses on the partial sequence the decoder has produced so far, cross-attention allows the decoder to compare this partial output with all encoded input tokens and select the most relevant information. Together, these mechanisms enable the decoder to generate coherent and context-aware outputs, such as summaries, reformulations, or answers to specific prompts

At the final stage, the decoder’s output is passed through a Linear + Softmax layer. The linear layer converts the decoder’s hidden state into a long vector containing one score for every word the model could generate. These raw scores do not yet mean anything on their own, so the softmax function is applied to turn them into usable probabilities [30, 52]. Softmax essentially looks at all the scores, compares them, and “smoothly” scales them so that the most likely words receive high probabilities while unlikely words are pushed close to zero. All probabilities then sum to one, making the choice interpretable. The model picks the word with the highest probability, adds it to the sequence, and repeats this process until the full output is produced [47].

This architecture enables transformers to generate fluent, context-aware language and makes them particularly well suited for tasks such as regulatory documentation,

where understanding nuanced relationships between domain-specific terms is essential [59].

Figure 2.2 provides a simplified overview of the Transformer architecture, showing the flow of data from input tokenization to output generation via encoders, decoders, and the final linear + softmax output layer [52, 62].

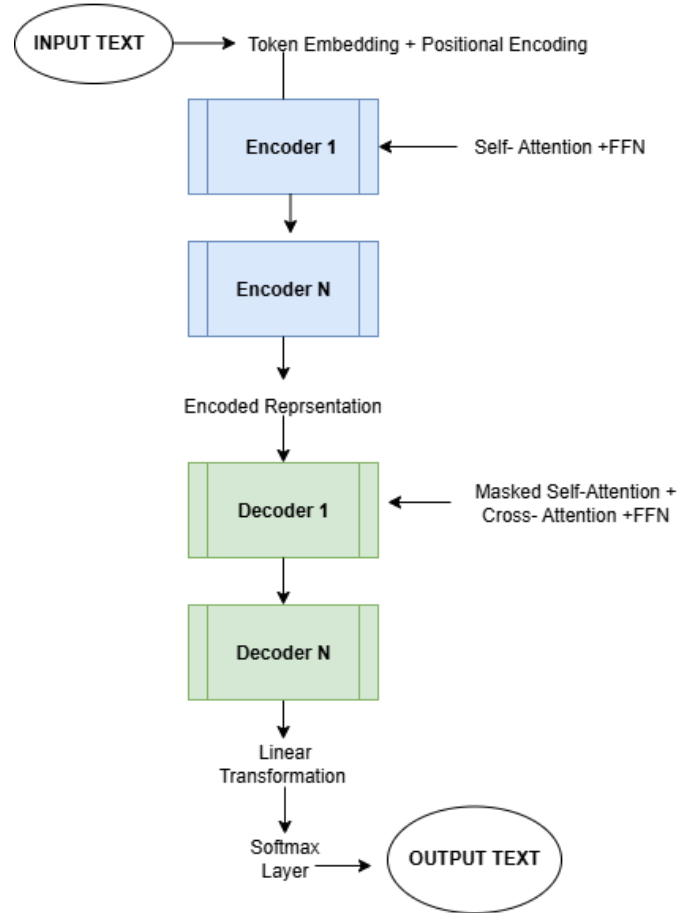


Figure 2.2: Simplified Transformer architecture

Prominent examples of such transformer architecture include Open-AI’s GPT series [4, 47, 48] and Google’s Bidirectional Encoder Representations from Transformers (BERT) [9], both of which have demonstrated the ability to model linguistic context, perform reasoning, and handle nuanced domain-specific language [52]. Even though they represent two foundational but distinct approaches to language modeling. Both utilize the transformer architecture introduced by Vaswani et al. [62], yet diverge in structure, training objectives, and downstream use cases [52].

Devlin et al. [9] introduced BERT as a transformer-based model that utilizes only the encoder component and is optimized through masked language modeling. This enables BERT to build rich, bidirectional representations by incorporating context from both the preceding and following tokens [9, 52]. As a result, BERT excels in language understanding tasks, such as classification, NER, and extractive question answering [9]. In domains like healthcare and medical risk documentation, BERT can be applied to extract relevant information from complex sentences, classify types of risks, and identify failure modes with high contextual accuracy [59]. The GPT, introduced by Radford et al. [4, 47, 48], is built exclusively on the decoder stack of the transformer architecture, specifically using masked self-attention and causal (left-to-right) processing to ensure that each token is generated based only on preceding tokens. The model follows an auto-regressive objective, meaning it estimates the probability of the next token while relying solely on the prior context [47]. This enables GPT to produce coherent, human-like text, making it highly effective for generative tasks such as summarization, drafting responses, or rephrasing regulatory statements [5]. Its ability to generate fluent language outputs makes it a strong candidate for creating or suggesting preliminary content in hazard documentation workflows [59].

These two models highlight the complementary strengths of transformer-based LLMs: BERT for comprehension and GPT for generation [52]. When applied to hazard key generation, such capabilities enable automated systems to both understand existing documentation and support the creation of new content [59].

However, a notable limitation shared by both models is their reliance on pretrained, static knowledge. Since LLMs are trained on large datasets before deployment, they lack real-time access to current or domain-specific data, a critical shortcoming in regulated fields such as medical device development, where safety standards, product data, and regulatory guidelines evolve continuously [43]. This makes it essential to supplement generative models with mechanisms that allow access to external, authoritative sources of information [35].

To address this, the next section explores RAG, a technique that integrates LLMs with retrieval components to ground outputs in up-to-date factual information [35].

2.1.2 Retrieval Augmented Generation

RAG, introduced by Lewis et al. [35], represents a significant advancement in the architecture of LLMs by incorporating external document retrieval into the text generation process. Unlike conventional LLMs, which generate responses based solely on their pre-trained internal parameters, RAG enables models to dynamically query and incorporate relevant information from external knowledge sources at inference time [35]. This dual-stage architecture addresses a critical limitation of standard LLMs, their static and closed knowledge base, which can become outdated or inadequate when dealing with highly specialized or evolving domains [3].

In domains such as medical device design and the preparation of risk management files, this functionality becomes exceptionally useful. These tasks require handling complex, technical, and regulatory texts that frequently evolve and demand high factual precision [59]. RAG’s architecture can be conceptually divided into two core stages. The first stage is the retrieval stage: the input query, such as a function, hazard, and cause prompt, is first encoded into a dense vector using a transformer-based neural encoder (*e.g.*, BERT [9]). This vector is subsequently matched against an indexed corpus through similarity search, often implemented with Facebook AI Similarity Search (FAISS), Meta’s open-source library optimized for scalable retrieval and clustering of dense vectors [29]. The top-k most semantically relevant documents are retrieved from sources. This retrieval step ensures the model accesses authoritative and up-to-date content tailored to the query’s context.

In the generation phase, the query and retrieved evidence are merged and processed by a generative decoder such as a *e.g.*, a GPT like model [4]. This decoder then produces a final output, such as a recommended safety message or explanation. As the generation is conditioned on both the retrieved knowledge and the input, it enables contextually relevant, traceable outputs that are better suited to regulated domains [35]. This architecture is illustrated in Figure 2.3.

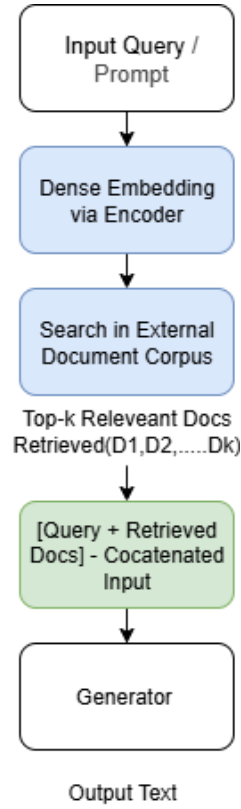


Figure 2.3: Overview of the Retrieval-Augmented Generation architecture.

In theory, integrating RAG into risk management workflows may reduce manual effort, increase consistency, and help maintain regulatory compliance by grounding outputs in validated content [59]. Moreover, its modular nature allows independent updates to the retriever or corpus without retraining the generator [35]. This makes the system adaptable to changing safety standards and device requirements.

However, RAG is not without its limitations. First, traditional retrieval techniques often rely on surface level embeddings and may miss nuanced or multi-hop reasoning as required for comprehensive analysis [31]. Second, concatenating long documents with queries risks exceeding input limits, potentially causing truncation or omission of important context [27]. These challenges are particularly pronounced in domains such as medical risk management, where traceability, clarity, and factual grounding are critical [41].

In summary, RAG introduces a flexible and dynamic framework that overcomes key limitations of LLMs by integrating external domain knowledge into the generative

process [35]. This retrieval-enhanced architecture holds promise for improving the quality, relevance, and accuracy of AI generated systems. While this thesis investigates RAG as a potential tool for improving risk control measures, its real-world effectiveness must still be evaluated empirically.

To address some of RAG’s current limitations, particularly in handling hierarchical documentation structures and ensuring deeper contextual relevance, the following section explores the RAPTOR architecture.

2.1.3 Recursive Abstractive Processing for Tree-Organized Retrieval

While RAG marked a major step forward in combining search and text generation, it still struggles with very long or complex documents. In particular, RAG retrieves a fixed number of short passages and combines them with a query, often leading to fragmented or incomplete answers [27, 35]. This is especially problematic in domains like healthcare or risk documentation, where relevant information is not always contained in one place but spread across multiple sections of a report. To address this, a newer framework called RAPTOR was proposed by Sarthi et al. RAPTOR improves both the structure and quality of information retrieval by organizing documents into hierarchical layers of meaning [53].

The main idea behind RAPTOR is to process documents in a way that reflects how humans often read complex material by first getting an overview, then narrowing down to specific sections, and finally focusing on exact details. To enable this, RAPTOR begins by breaking a document into small, readable segments, usually a few sentences long. These segments are then compared to one another to see which ones talk about similar topics. Instead of doing this manually, RAPTOR uses a model trained to understand language similar to how translation apps or chatbots work to assign each segment a kind of “meaning fingerprint”. These fingerprints are then grouped into clusters that contain similar content [53].

Each group of related segments is summarized into a short paragraph using a LLMs. For example, if several segments describe steps in a medical safety protocol, the system generates a concise summary of the entire process. These summaries are again grouped and summarized, creating a layered structure a bit like a tree with

branches and leaves. At the bottom, you have the original text segments. In the middle, you have summaries of related content. At the top, there is one or more high-level summaries that give a general understanding of the whole document. This is why the method is called “recursive”, it repeatedly summarizes, building a tree of meaning [53].

When a user submits a question, RAPTOR searches this tree instead of scanning all the raw text at once. It can either move through the tree step by step, starting with the most general summaries and then diving deeper, or it can search across the whole tree at once to find the most relevant pieces of information. This flexibility helps the system answer questions more accurately, especially when the needed information is spread out or is only indirectly mentioned. It also reduces the risk of overwhelming the language model with too much text at once, which can cause confusion or hallucination [53].

In tests on benchmark datasets that evaluate how well models can answer questions based on long documents, RAPTOR showed strong improvements, including gains of up to 20% in absolute accuracy. For instance, in a dataset called QuALITY, which includes long reading comprehension tasks, RAPTOR combined with GPT-4 performed significantly better than traditional methods. It also achieved top results in tasks focused on understanding scientific papers and fictional stories. These improvements suggest that RAPTOR structure helps language models handle complex reasoning and remember important details more effectively [53].

However, RAPTOR is not without limitations. The way it groups and summarizes information depends on statistical assumptions that may not generalize equally well to highly diverse or technical documents. When clustering is less accurate, summary quality can degrade [33, 39]. Additionally, building and maintaining the hierarchical structure can be computationally expensive, especially for large document collections, making scalability a practical concern [53]. Researchers are exploring ways to improve this by using more flexible grouping strategies and by adapting the structure dynamically as new information becomes available.

Overall, RAPTOR represents an important advancement for retrieval augmented systems. Mimicking the way people read and summarize documents, it allows AI models to navigate complex material more intelligently. This makes it especially relevant for applications that involve long, structured, or regulation-heavy documents,

such as risk management documents in healthcare, where precise understanding and clear reasoning are critical.

2.1.4 Discussion and Intermediate Summary

Section 2.1 addresses RG1 and, more specifically, RGO1.1 by introducing the technical building blocks that are relevant for this thesis. First, different types of GenAI models were outlined. This subsection showed that, while image and signal-focused models are useful in many settings, this thesis is centered upon LLMs. The section then explained how LLMs process text using the transformer architecture. Tokenization, embeddings, self-attention. The encoder-decoder structure was described to clarify how these models can understand and generate complex technical language. The distinction between encoder-based models such as BERT and decoder-based models such as GPT is important for this thesis. BERT type models are strong in understanding existing documentation, while GPT-type models are strong in generating new text. For hazard key generation, both abilities are relevant: the system must read and align with existing safety messages, and also suggest new suggestions when needed. However, the section also showed the main limitation of standalone LLMs: they rely on static, pre-trained knowledge and cannot automatically stay in sync with a specific, evolving hazard key dataset. For this reason, retrieval-augmented methods such as RAG and RAPTOR were introduced. RAG allows an LLM to retrieve relevant documents at query time and generate answers that are grounded in this retrieved content. RAPTOR extends this idea to long and structured documents by building a hierarchy of summaries, making it easier to find the right information in complex material.

For the system developed in this thesis, these insights have direct consequences. The chatbot will not use a free-running LLM. Instead, it will follow a retrieval-first approach. The model will work on top of an internal hazard key dataset and use retrieval to anchor its outputs in existing safety messages. The model resorts to generation only in cases where no sufficiently relevant match is identified. Ideas from the RAG and RAPTOR guide how retrieval and generation are combined.

There are still some Open Questions (OQs) about the approaches that were looked at. (OQ01) retrieval-driven systems only work well when the datasets are complete and kept up to date. (OQ02) embedding and clustering methods are still sensitive

to parameter choices and can be affected by changes in the domain or terminology. (OQ03) even with retrieval augmentation, hallucinations can't be completely gotten rid of, and explainability isn't as good as regulatory-grade traceability. Also, most methods don't have ongoing validation or monitoring in the real world, which makes it hard to know how reliable they will be in the long run. Whether and to what extent these open points can be addressed will be discussed in Chapter 6.

2.2 Risk Control Workflow – Hazard Keys

This section addresses RGO1.2 by providing a detailed overview of how a hazard key is structured, documented, and communicated within the product development and documentation life cycle at Siemens Healthineers. The formulation of risks in medical device documentation follows a clearly defined workflow that is embedded within the broader risk management plan. This process is not only crucial for product and patient safety but also plays a central role in fulfilling regulatory, usability, and internal traceability requirements.

Moreover, the workflow ensures that safety messages are accurately communicated to the end user and linked to internal risk analyses and quality assurance processes. A precise understanding of key terminology, such as risk, hazard, and hazard key, is essential, as these terms are formally defined in both international standards and internal documentation practices. Understanding this workflow forms the foundation for designing the AI system for hazard key generation.

This section covers the following aspects, subsection 2.2.1 the definition of key terms, subsection 2.2.2 how risks are formulated and communicated in user documentation, particularly in the form of standardized safety messages, subsection 2.2.3 a discussion and intermediate summary that connects the current workflow to the motivation of this thesis.

2.2.1 Definition of Key Terms

It is important to define the key terms that are used throughout the risk management process before talking about how hazard keys are made and used. Siemens Healthineers uses these ideas to figure out how to evaluate risks, write down safety

measures, and connect them to user documentation [64]. Knowing them also helps explain why there is a structured system like the hazard key and how a GenAI model can help with this process.

Risk

Risk is defined as the combination of a hazardous situation, the severity of the harm it may cause, and the probability of occurrence of that harm [24].

Severity is categorized into four classes:

- **Class 1 (Negligible)**: Minor injury or temporary discomfort.
- **Class 2 (Moderate)**: Injury requiring medical treatment.
- **Class 3 (Critical)**: Serious injury or death of an individual.
- **Class 4 (Catastrophic)**: Serious injury or death of multiple individuals

The Probability of Occurrence is rated from A (inconceivable) to F (frequent). For instance, “frequent” (F). For instance, in the Siemens Healthineers risk management framework, “frequent” (F) is defined as more than 100 occurrences per 1,000 units per year, based on estimated or historical data [24].

Risk Control Measures (RCMs) are actions taken to reduce the likelihood of harm or lessen its severity. They are based on both historical data and risk assessments performed during development and verification. Assessing both the likelihood and the severity of possible hazards allows risks to be quantified and effectively controlled [64]. This allows RCMs to focus on reducing either how likely the risk is to occur, how serious the outcome would be, or both. Common strategies include design changes, protective hardware, software safety checks, user training, and warnings in the documentation.

At Siemens Healthineers, according to the risk management plan [64], RCMs can be addressed through direct implementation (*e.g.*, design modifications) or modulation (*e.g.*, procedural adjustments). If these measures are not feasible or cannot be applied immediately, the remaining option is to create a safety message, which is then included in the hazard key list. Therefore, in this context, we use the term safety message to refer to the output or expression of the risk control measure.

Hazard and Hazard Key

A hazard is defined as a potential source of harm [1, 7]. In medical contexts, typical hazards may include but are not limited to radiation, electrical current, incorrect drug dosage, or misinterpretation of imaging data. A hazardous situation is defined as the specific combination of circumstances that creates exposure to a hazard for persons, property, or the environment. For instance, while “radiation” is the hazard, a “patient exposed to unshielded radiation” constitutes a hazardous situation [7].

Examples from Annex B of IEC 62366-1 illustrate common hazards such as radiation energy (which may cause burns), sharp needles (which may introduce infections), and incorrect drug administration (which may lead to respiratory arrest or paralysis). To ensure structured documentation and traceability, Siemens Healthineers employs a formalized system of hazard keys.

A hazard key is a standardized identifier that encodes a function, hazard, cause, and safety message into a single string, ensuring end-to-end traceability across risk analysis, product design, technical documentation, and regulatory compliance workflows. Hazards are identified early in the development lifecycle, based on clinical workflows such as scanning, reconstruction, and image display, as well as system functions and foreseeable misuse. Once identified, each hazard is linked to a unique key that supports traceability throughout the entire risk management process.

The structure of a Hazard Requirement Key (HRK) typically follows the syntax:

- F01 indicates the function: *e.g.* **Patient Registration**
- G01 represents the hazard: *e.g.* **Incorrect Diagnosis**
- U03 shows the cause: *e.g.* **Wrong Entry of Patient Orientation**
- M03 defines the mitigation: *e.g.* **A Warning in the Operator Manual**

For example, the key hz_ga_F01G01U03M03 encodes a case in which a wrong patient orientation entry during registration leads to an incorrect diagnosis, and the chosen mitigation or safety message is to provide a warning in the user manual.

These structured identifiers are used extensively in internal systems such as publication of safety relevant content.

Hazard keys serve as anchor points for collaboration across engineering, quality assurance, and technical communication teams. Moreover, because each hazard key encodes a complete risk narrative from system function to mitigation, it becomes machine readable and serves as an ideal entry point for automation through generative AI technologies. This structured format not only ensures consistency and regulatory compliance but also supports the integration of AI systems capable of retrieving, interpreting, and generating safety documentation based on risk management logic [24, 57].

Hazard keys are important because they ensure clear traceability from the initial risk analysis to the implementation and final documentation of risk control measures. They serve as a link between the identified risks and the corresponding safety messages that appear in documents such as the Instructions for Uses (IFUs).

By doing so, they help to ensure that each risk is properly addressed, mitigated, and verified. This traceability is essential to meet regulatory requirements and demonstrate compliance Component Oriented Structured Information Management Application (COSIMA), the company's content management platform, which governs modular authoring and enduring audits or assessments.

Hazard keys also support consistency across teams by providing a common structure that connects risk analysis with product development, quality assurance, and technical writing. These hazard keys are used in several critical areas. They are documented in the Risk Management Plan [64] and Risk Analysis to trace how specific hazards are managed. They also appear in technical documentation such as IFUs and service manuals to communicate safety-relevant information to end users. A clear representation of the terms is shown in the figure 2.4

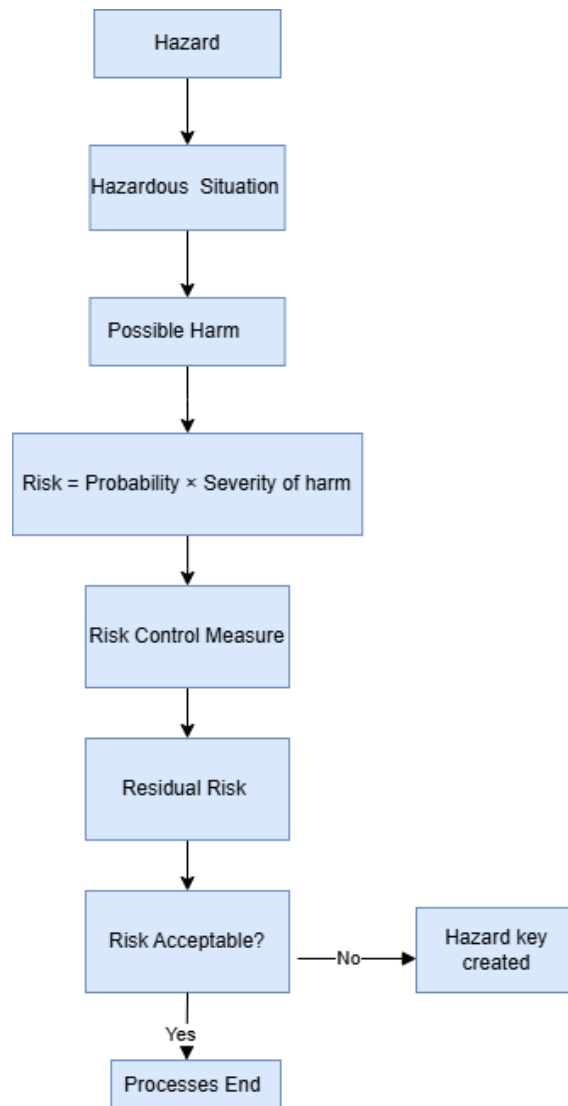


Figure 2.4: Flowchart to represent the terms

Internally, they are managed through systems like COSIMA, Siemens Healthineers' content management platform, which is used to author, track, and publish standardized safety content across different product teams. Through this integrated use, hazard keys play a central role in ensuring safety, traceability, and compliance across the product lifecycle.

2.2.2 Risk Communication in User Documentation

According to IEC/IEEE 82079-1 [23], effective risk communication in user documentation must be accurate, well structured, and context-sensitive. These principles are applied through a combination of standardized message types, structured formatting, and internal authoring practices that ensure both regulatory compliance and usability.

Siemens Healthineers classifies safety-related documentation into the following categories:

- **Safety notes:** General warnings placed at the beginning of documents or chapters.
- **Warning messages:** Step specific alerts are shown just before describing a potentially hazardous operation.
- **Safety signs and labels:** Pictograms or text warnings displayed on the device or in the user interface.

Each safety message follows a three-part structure:

- **Source:** The condition or user action that creates the hazardous situation.
- **Consequence:** The possible harm if the hazard is not avoided.
- **Countermeasure:** A specific instruction on how to prevent the harm.

These components are preceded by a *signal word*, which indicates the severity of the risk:

- **DANGER:** Will result in serious injury or death.
- **WARNING:** Could result in serious injury or death.
- **CAUTION:** Could result in minor or moderate injury.
- **NOTICE:** Not related to safety, but relevant to the operation of the product.

Placement of safety messages follows internal guidelines. Safety notes appear at the beginning of documents, while warning messages are positioned just before hazardous steps. In Graphical User Interfaces (GUIs), on screen warnings remain visible as long as the hazard exists and often require user confirmation.

The Siemens Authoring Guide [57] specifies that only trained personnel may create or edit safety messages. Each message must be traceable to a hazard key and authored using COSIMA, the internal documentation platform. This tool ensures consistency in formatting and integration with translation and risk analysis workflows. Safety message wording follows a “telegraphic style,” using concise, imperative, and unambiguous language (*e.g.*, “Use protective gloves.” rather than “You may want to use gloves.”).

A recurring challenge in the documentation process is the manual creation of compliant safety messages from newly identified hazards, often triggered during product development or in response to customer feedback. This process is labor-intensive and demands considerable cognitive effort to maintain consistency and comply with regulatory requirements.

This thesis hypothesizes that GenAI, specifically LLMs, can assist in the authoring of structured, consistent, and traceable safety messages. Such automation has the potential to reduce the workload, improve documentation quality, and accelerate development cycles without compromising safety or compliance.

2.2.3 Discussion and Intermediate Summary

This section provides the methodological and organizational background needed to understand how hazard keys are created and used at Siemens Healthineers, where we are addressing RGO1.2. It explains the key terms and definitions of the risk management workflow. It shows how risks are evaluated for the particular hazard and hazardous situation. Once hazards are identified and risks are evaluated, not all risks need to be mitigated; some can be implemented with or without RCM, but when mitigation is required, that’s when safety messages are formulated and outsourced with hazard keys

This manual process outlines the functional requirements that the chatbot must be able to support. By understanding how hazard keys are currently created, we

can make sure that the AI system fits naturally into the existing process, meets compliance expectations, and supports users in creating accurate and traceable safety documentation.

There are still two OQ: First, (OQ04) the system needs to be able to deal with cases where the same Function–Hazard–Cause (FHC) input leads to more than one valid safety message. In these situations, the chatbot should show the expert all of the possible options so they can choose the right one and give it the right HRKs. Second, (OQ05), the system needs to be able to find inputs that are missing or not in the right format and respond appropriately. This could mean asking for more information or stopping the retrieval process until all fields are in the right format. This makes sure that only complete and correct entries move on in the workflow.

2.3 Data Protection and Regulatory Requirements for AI Systems

The development and deployment of AI systems in healthcare, particularly those designed for safety documentation, are governed by a set of legal, regulatory frameworks. These frameworks collectively ensure that AI applications safeguard patient privacy, uphold standards of safety, and foster transparency and accountability throughout their lifecycle.

This section addresses RQ2 and the objectives of RGO1.2 by examining the primary regulatory instruments relevant to such systems in detail. This section is as follows: subsection 2.3.1 GDPR, subsection 2.3.2 EU AI ACT, and, where applicable. In addition, it outlines the foundational principles of Trustworthy AI in subsection 2.3.3, which serve as an ethical compass for responsible design and operation. The analysis not only explains the scope and requirements of each framework but also assesses, in realistic terms, their applicability to the proposed chatbot, which is clearly mentioned in subsection 2.3.4.

2.3.1 General Data Protection Regulation (GDPR)

The GDPR (Regulation European Union (EU) 2016/679) governs how personal data is handled in the EU. The prototype described here does not process personal, clinical, or operational data. It works only with structured, anonymized hazard data, simple FHC entries, and predefined safety messages. Because no personal information is involved, the strict GDPR requirements do not apply, but its core principles still shape the system’s design [63].

The scope of the system is deliberately narrow. It retrieves or generates safety messages from a small, curated dataset, and the purpose of this processing is documented in a basic register [49]. Expanding the system to include incident reports or device-specific logs would require new legal grounds and clearer transparency measures, as noted by Voigt et al. (2017) [63]

Data minimization is built in: the dataset contains only the data needed for the task, and the schema is fixed ahead of time. Because the data is manually checked and does not change often, accuracy and consistency can be maintained with simple validation steps.

Even without personal data, standard safeguards are used. Access to datasets and model checkpoints requires authentication, and development and testing environments are separated to prevent accidental exposure [49]. Logging access and processing events supports traceability, and more formal retention rules could be added if the system ever handled sensitive material.

Version control for datasets, prompts, and model checkpoints provides an audit trail. Any output can be traced back to the exact model state and data source. Although accountability obligations are minimal in this research setting, keeping these records now makes it easier to scale governance later. Article 9 of the GDPR places strict limits on processing health data. The system avoids this entirely by design. A full

Data Protection Impact Assessments (DPIA) is not required at this stage, but carrying one out early can help identify risks if future updates introduce richer or more sensitive datasets [14].

2.3.2 EU Artificial Intelligence Act (AI Act)

The EU AI ACT is the first set of rules in the European Union that covers all aspects of the development and use of AI systems. It extends to all businesses that provide or operate AI systems in the EU, even if the company operates from outside the EU. The Act broadly defines AI (Article 3), including statistical, rule-based, and machine learning methods [11, 50]. The Act’s risk-based classification system is a key part. This divides AI systems into risk categories: unacceptable, high, limited, and minimal.

The system’s potential effects on health, safety, and basic rights raise the bar for obligations [11]. High-risk systems, like diagnostic support tools or AI used in critical infrastructure, have to follow strict rules. These rules include having a written risk management process (Article 9), quality-controlled datasets (Article 10), detailed technical documentation (Article 11), logging (Article 12), transparency obligations (Article 13), meaningful human oversight (Article 14), and protection against errors and cybersecurity threats (Article 15). When an AI system affects clinical decisions or patient outcomes in healthcare, it is usually classified as high-risk.

The prototype created in this study does not belong to this category. It works with a set of pre-defined Hazard Requirement Keys (HRKs) and safety messages. It doesn’t do any diagnostic work or get in the way of clinical workflows. The system instead gets or makes safety messages that a human expert must always check and approve. The system fits into the limited-risk category under the Act because it doesn’t work on its own or change how patients are cared for.

Article 52 says that limited-risk obligations mostly have to do with being open and honest. In practice, this means making it clear to users that they are using an AI system, telling them what it is supposed to do, and listing any known problems with it. For this prototype, these requirements mean showing system disclaimers, making it clear that the outputs come from a controlled hazard dataset, and making it clear that the chatbot does not replace expert judgment.

For this project, these kinds of practices include keeping track of where datasets come from, keeping audit logs, making sure that every output is checked by a person, and keeping the processing environment safe. These steps make the system more reliable and easier to understand without adding extra rules. However, full traceability is still hard for LLM based components because their internal reasoning can’t be fully

reconstructed [58]. In short, the EU AI ACT gives a fair framework that fits well with this research. The prototype is clearly in the low-risk group, which means that it needs to be open and honest in its communication. It also stays flexible in case its scope grows to include more regulated applications in the future.

2.3.3 Trustworthy AI

The European Commission’s High-Level Expert Group on Artificial Intelligence (HLEG AI) defines trustworthy AI as systems that are legally compliant, ethically sound, and technically dependable throughout their entire lifecycle [12, 36]. The framework puts these expectations into seven groups, which together show what responsible AI design should look like in real life.

Human agency and oversight stress that AI should help, not take the place of, expert judgment. Users need to know that they are dealing with an automated system and should still be able to question, fix, or change its outputs. This principle is directly applied to this chatbot: a domain expert must review and approve every safety message that is retrieved or created before it can be used.

Technical robustness and safety have to do with how reliable a system is in both normal and unexpected situations. This includes protections against mistakes, models that don’t work right, or inputs that are meant to hurt the model. For the proposed system, robustness mainly means getting the right hazard message every time and not making any unintended model extrapolations when there isn’t an exact match.

Privacy and data governance make sure that data are used in a way that is safe and in line with laws like the GDPR. No personal data is involved in this prototype because it only uses structured, anonymized hazard data. Still, following the principle means keeping access under control, keeping track of different versions of the dataset, and making sure that updates don’t cause quality problems or biases [63].

Being transparent means being able to tell how the system works. For this project, transparency is achieved by clearly showing whether a message was retrieved from the database or generated by the LLM, presenting the linked hazard key, and logging outputs for later review. This lets people see how the final safety message was made and decide if it was appropriate.

Diversity, non-discrimination, and fairness mean stopping the system from acting unfairly or unequally. Even though the dataset here doesn't include demographic or sensitive information, fairness is still important to make sure that the quality of the messages is the same across all functions, hazards, and causes, and to avoid systematic gaps that the model's generation process might create [15].

Societal and environmental well-being emphasizes that AI systems must yield beneficial outcomes and avert inadvertent harm. For this system, this means making sure that the automated help makes safety documentation more consistent and efficient without sending the wrong messages or making people too dependent on automation.

Finally, accountability means that roles and responsibilities are clearly defined and that you can check how the system is working. The prototype has controlled datasets, logs of all retrieval and generation events, and documented model prompts and checkpoints to back this up. These artifacts enable reviewers to reconstruct the process of output generation and evaluate compliance with governance requirements [16, 20].

These principles support the main design choices for the proposed chatbot: limited input data, clear steps for retrieving and generating data, and the need for human validation. Not all Trustworthy AI requirements are fully applicable because the system is limited in scope. However, incorporating the relevant elements establishes a robust ethical and operational foundation for a future production version in a regulated healthcare setting [36].

2.3.4 Discussion and Intermediate Summary

Section 2.3 outlines the main rules that are relevant for the chatbot. The system only works with structured input functions, hazards, causes, and safety messages, and it never uses personal or patient data. Because of that, GDPR obligations are limited. What matters is keeping the current safeguards: using only anonymized inputs, restricting the dataset to approved hazard keys, and logging all interactions.

These measures support purpose limitation, data minimization, and accountability, even though the system is only a research prototype. Full GDPR compliance is not required here. Under the EU AI ACT, the system falls into the limited-risk category

because it does not influence clinical decisions and only works with predefined safety messages. It must tell users that they are interacting with AI, state its purpose, and be clear about its limits. Human oversight is still needed, meaning a domain expert reviews and approves every retrieved or generated message. Trustworthy AI principles also help guide the design. For this project, the key points are transparency, expert supervision, and technical reliability. These are already built into the workflow: the system does not create medical content on its own, outputs are checked against existing hazard keys and templates, and humans carry out the final validation. Fairness or robustness testing is unnecessary because the dataset is fixed, structured, and not personal.

Overall, the regulatory and ethical considerations support the existing design choices: limit inputs to structured hazard data, keep human reviewers involved for all safety-related outputs, maintain complete logs for traceability, and make the system's role and boundaries clear to users.

2.4 AI Governance Frameworks and Compliance Strategies

This section puts RGO2.2 into action by turning governance concepts into functional, occasionally experimental, controls used in the GenAI safety documentation chatbot. The focus is on looking at what can be done given the limits of current model capabilities, healthcare laws, and available resources, rather than claiming full compliance readiness. AI governance frameworks set up the rules and procedures that AI systems must follow to work safely, responsibly, and in accordance with the law. Compliance techniques turn these ideas into real actions [16, 37, 51].

2.4.1 Hourglass Model

Mäntymäki et al.'s [37] Hourglass Model is a useful way to connect big governance goals to technical implementation. The GDPR, the EU AI ACT, and the European Health Data Space (EHDS) are all examples of frameworks that specify the general constraints for how the system must work [13, 49, 50]. These criteria had a big impact on many of the first design choices. For example, they limited the dataset to

anonymised hazard data and made it necessary for a person to look at all the results. We have gone over these regulations in depth in Section 2.3.

The “organizational layer” is about how things are run and who is in charge of making sure they run well. In this prototype, these aspects are not built into real organizational processes; instead, they are emulated through controlled workflows [45]. Even though this research doesn’t look at how these kinds of processes might operate with clinical governance structures directly, they show how they might work in a prototype setting.

The “system layer” is where the true technological protections that make governance principles operate come into play. This includes mechanisms to keep track of things, safe logging, support for explainability, and regulation of I/O. These characteristics indicate what governance can look like at the system level, but they also show that there are still concerns, including not being able to adequately explain things and the chance of hallucinations [40].

Governance-by-Design: The system follows the ideas of “privacy by design” and “accountability by design” throughout its lifecycle [63]. This means that built-in risk checks, traceability, and access controls are there from the start. For the chatbot, this involves using only anonymized hazard information, limiting access through role-based permissions, and logging all retrievals and generated outputs. A formal DPIA is not required for the prototype, but doing a DPIA style review is still useful for spotting future risks, for example, if the system were expanded to handle incident reports or device-level identifiers [8]. These steps help set a basic governance baseline, even though the system is not ready for regulated clinical use. Some issues remain: the prototype does not cover every clinical scenario, LLM hallucinations are still possible, and operational risks cannot yet be monitored in real time. Governance-by-design cannot remove these challenges, but it provides a clear way to handle them gradually.

Transparent Decision Support: aims to ensure that system outputs can be explained, traced, and reviewed. [40]. This matters in safety-critical work, where users must understand both the content of a message and its source. The system supports this process by using RAG and RAPTOR [35, 53]. When an exact match exists in the structured hazard database, the chatbot retrieves the safety message and shows

the linked hazard key [65]. If no match is found, the LLM produces a provisional message, and the interface indicates that it was generated rather than retrieved. This distinction helps users judge how trustworthy the output is and whether it needs editing.

Even with these measures, a full explanation is not achievable. Retrieval improves traceability only to a degree, and the internal reasoning of an LLM remains opaque. Regulatory level transparency cannot be guaranteed at this stage. The prototype's contribution is therefore incremental: it shows how transparency can be strengthened within the limits of the available methods.

Clinical Oversight and Risk Escalation: Under the EU AI ACT and medical device governance rules, clinical oversight ensures that human experts remain responsible for all safety-related decisions [50]. In this prototype, oversight works by flagging outputs with low retrieval confidence, unclear FHC entries, or deviations from the predefined safety-message templates. A domain expert reviews and approves every output, whether it comes from the database or is generated by the LLM. As a result, the system functions only as decision support and has no contact with patients or clinical workflows.

The limitation is that this oversight setup is used only in a research context and is not part of any real clinical or organizational process. Still, it illustrates how a human-in-the-loop approach could operate in a deployed system, where expert review would remain essential for safety.

Operational Controls and Technical Safeguards: Operational controls are needed to protect an AI system and prevent misuse. In the current prototype, only the developer has access, and the system runs in a separate research environment with no connection to clinical or organizational systems. Because of this setup, full security features such as Role Based Access Control (RBAC), Multi-Factor Authentication (MFA), and containerized deployment are not yet implemented, though they will be essential if the system is ever used outside a research context.

The main limitation is that the security measures reflect a prototype rather than a production system. The points described above act as a plan for future development

and show how operational controls and technical safeguards would need to evolve before the system could support more advanced or real-world use.

Auditability and Continuous Validation: For the system to be auditable, its actions must be traceable and easy to reconstruct. In the prototype, this is done by logging all interactions, retrieval attempts, safety messages, and generated hazard keys. Each output is reviewed and approved by the user, and that confirmation step is logged as well. This step makes it possible to see how a message was produced and whether it was accepted or corrected.

However, this approach only works for single interactions. It does not meet regulatory expectations for continuous validation, which requires long-term performance monitoring, drift detection in real deployments, and structured testing across different clinical environments. Because the system exists only as a research prototype, full lifecycle monitoring is not yet possible. Even so, the current logging and manual review process provides a useful foundation. It shows how iterative auditing could function in a future operational system and could be expanded into full continuous validation if the system moves toward regulated use.

Lifecycle Governance and Supply Chain Integrity: Lifecycle governance ensures that an AI system and its components remain reliable, safe, and documented throughout development, deployment, maintenance, and retirement. Supply-chain integrity extends this to the data, models, and software the system depends on, requiring checks on their origin and trustworthiness [25].

The current prototype applies these ideas in a simple, research-focused manner. Data sources are checked manually, and the origin of the structured hazard dataset is tracked to confirm that all entries come from approved internal material. Basic logs record how data and model elements are used during development, allowing unusual behavior or issues to be identified and reviewed. These steps do not yet form a full supply-chain governance framework, but they show how such controls could be built in a more mature system.

The main limitation is that these measures have not been tested in a real operational setting and do not cover the broader range of dependencies and third-party components a production system would require. In this prototype, lifecycle governance

serves as a starting point sufficient for research but needing much more detail and formal structure before it could support regulated healthcare workflows.

2.4.2 Discussion and Intermediate Summary

This part shows how RGO2.2 can be put into practice by turning general governance ideas into concrete safeguards for the safety documentation chatbot. The Hourglass model was responsible for this model by focusing on system and organizational levels. The broader legal setting was already discussed in Section 2.3. The section walked through several practical points: building safeguards directly into the system, using RAG for clearer decision support, adding a basic form of clinical oversight, putting simple operational checks in place, keeping logs for later review, and taking first steps toward lifecycle and supply-chain checks. These measures illustrate how accountability and traceability can be added, even in a small research prototype.

The limits of the system are also acknowledged. It cannot fully explain its reasoning, it cannot prevent all hallucinations, and it is not developed enough to meet regulatory requirements. Because of this, the controls presented here should be considered early experiments rather than final solutions.

In short, governance in a prototype grows step by step. The measures described here provide a starting point and show how technical safeguards and organizational needs can be linked, giving a direction for how validation and monitoring might develop later.

2.5 Complete Summary and Discussion

Chapter 2 outlined the key technical, procedural, and regulatory points needed for a GenAI-based system that supports hazard key creation. Section 2.1 explained how GenAI, preprocessing, clustering, and RAG/RAPTOR can help generate structured hazard keys, while also noting current limits, such as hallucinations and the need for clean input. These findings shape RG1 and RGO1.1.

Section 2.2 described the existing Siemens Healthineers workflow behind RGO1.2. It ensures traceability and compliance but is time-consuming and depends on expert

review. This part makes clear which tasks GenAI can support and which must stay under human oversight.

Section 2.3 covered the legal and ethical requirements for AI in healthcare (RGO2.1). Since the prototype only uses anonymized hazard data, not all obligations apply yet, but the section shows what would change if real data were used.

Section 2.4 translated these requirements into practical governance steps for RG2 and RGO2.2, including transparent retrieval, clustering support, and structured logging. These measures are minimal but suitable for a research prototype.

In short, the chapter defines what is realistically possible and which limits must be acknowledged (OQs). The prototype will use RAG/RAPTOR, clustering, and linked sources to support hazard key creation, while accepting that hallucinations and full regulatory compliance cannot be fully solved at this stage. This groundwork leads into Chapter 3, where the system model and architecture are developed.

3 Design and Concept Modeling

This chapter details the design and conceptual modelling of the chatbot. Building on the regulatory and governance foundations established in Chapter 2, the focus here shifts from principles to practice, showing how requirements are translated into design decisions.

The chapter follows a user-centered and governance-aware approach, ensuring that system development is both aligned with user needs and compliant with regulatory constraints. In doing so, it directly addresses RGT1.1 and RGT1.2, which concern the operationalization of trustworthy design methods and their integration with governance mechanisms.

The structure of the chapter is as follows: Section 3.1 introduces the design method guiding this work, Section 3.2 defines the context of use, Section 3.3 develops the use case models, Section 3.4 presents the design solutions. The chapter ends with Section 3.5, which reflects on the design insights and their implications for the subsequent implementation in Chapter 4.

3.1 Design Method

The design work in this thesis follows the User-Centric System Design (UCSD) framework, originally introduced by Norman and Draper [42] and later formalised by Mayhew [38]. UCSD uses an iterative cycle of design, testing, and refinement to ensure that system decisions are based on real user needs and the actual context of use.

Figure 3.1 shows the original UCSD model. It starts with understanding the problem space and user requirements, followed by developing and reviewing design concepts. Every step involves evaluation, and new information is used to inform subsequent

iterations. Deployment is not the end of the process, but another point for feedback and improvement.

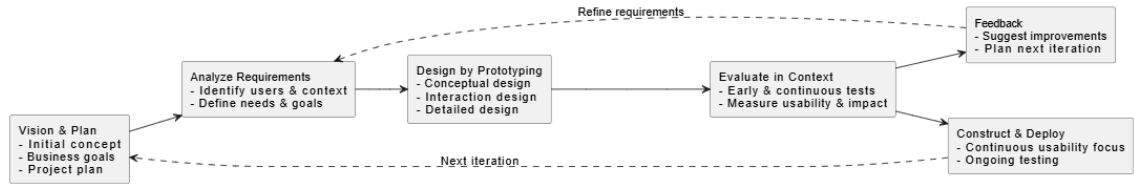


Figure 3.1: UCSD framework adapted according to Mayhew [38]

This thesis adapts the framework to suit a regulated healthcare context, prioritizing user needs while also addressing safety, compliance, and governance concerns. As seen in Figure 3.2, the process starts with the context of use: identifying the users, the tasks they undertake, and the system’s integration inside current workflows. Representative use cases are developed to delineate functional needs and anticipated interactions.

Design solutions are then modeled from several viewpoints, including information structure, interaction flow, system logic, user interface, and architecture. The models are integrated into a cohesive prototype that can be evaluated promptly by domain specialists. Every iteration concludes with a concentrated assessment session. Usability and safety considerations are evaluated by representative users and domain experts. Their feedback influences subsequent iterations and directs design modifications. The modified UCSD workflow facilitates a systematic refining process that enhances both usability and compliance.

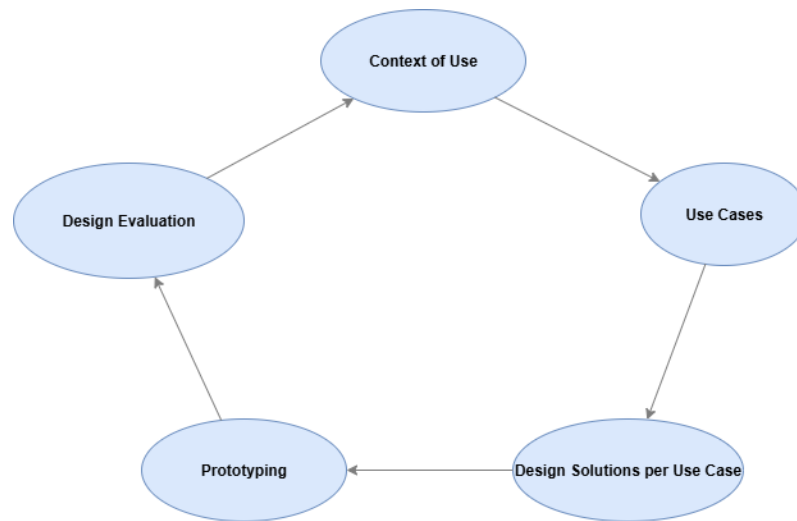


Figure 3.2: Adapted UCSD workflow applied in this thesis

3.2 Context of Use

This section outlines the operational context in which the prototype is intended to operate and clarifies its role within the broader user-centered design approach outlined previously. The focus is on a chatbot that supports the generation of safety messages and Hazard Requirement Keys (HRKs) for CT systems.

The trigger for entering the risk management evaluation can come from various sources, including customer complaints, voluntary or mandatory incident reports, user misunderstanding, newly introduced or modified functions, detected defects, or workflow changes; however, none of these triggers automatically create a hazard key. It only initiates a review to determine whether a formal risk assessment is necessary. If an assessment is necessary, it adheres to a defined structure that includes the identification and evaluation of risks, sequences of events, hazardous situations, and potential harms (probability and severity). Once the risk is understood, risk management determines whether it can be reduced to an acceptable level, and if so, appropriate RCM are implemented, which may include design changes, technical solutions, protective measures, or safety information such as warnings or training. If these RCM can be fully implemented and reduce the risk to an acceptable level, no hazard key is required. Sometimes risk can be accepted with or without RCM also.

A hazard key is only created when the identified risk cannot be eliminated or reduced to an acceptable level by a RCM, or if the required mitigation cannot be implemented in time, and therefore, user awareness becomes necessary. In these cases, the safety message and hazard requirement key are assigned. This process is illustrated in Figure 3.3.

This process is time-consuming, since each step depends on expert input, iterative discussion, and manual documentation. Moreover, it is not a one-off task but an ongoing activity across the entire product lifecycle. Every new function, design change, defect, or customer feedback may trigger the need for reassessment, and, where necessary, mitigation could be required.

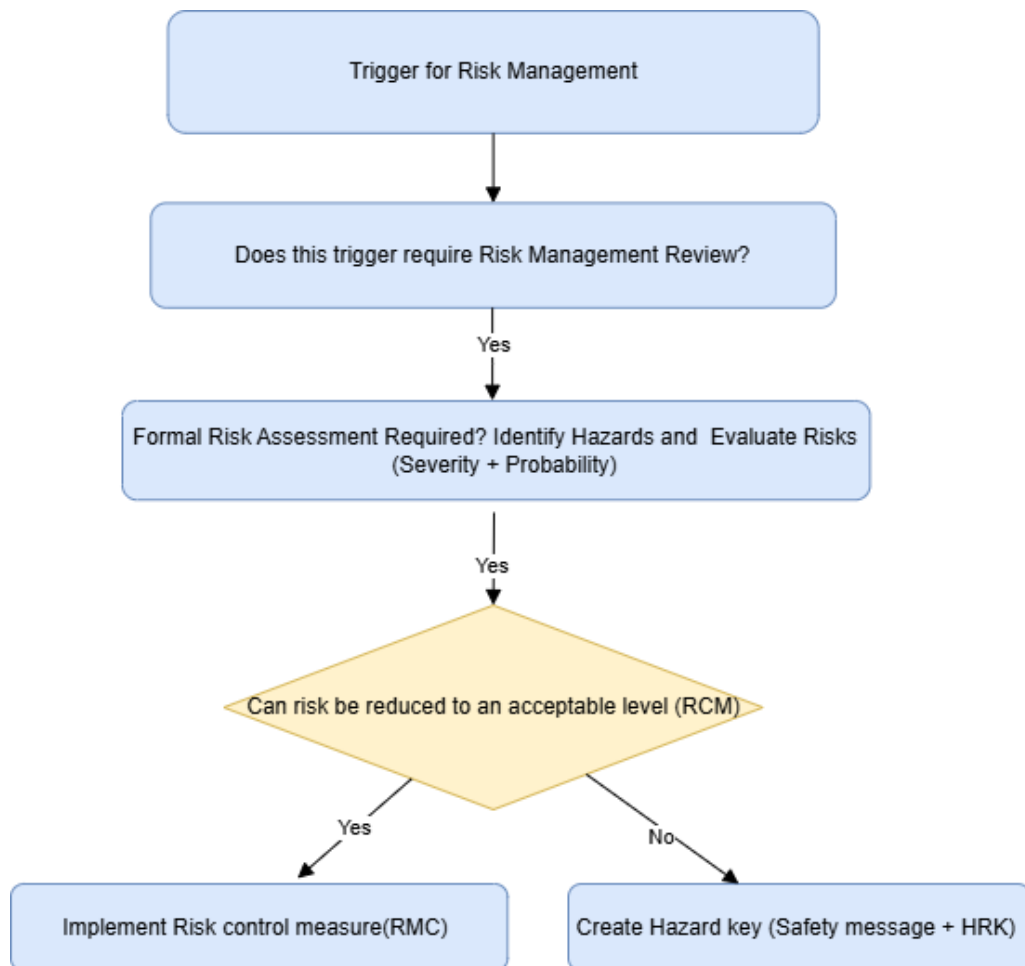


Figure 3.3: Manual Workflow for Hazard key creation

The prototype is designed to support this process of creating hazard keys. The chatbot receives structured inputs. The system then generates a candidate safety

message. The expert reviews the proposed message and decides whether to accept or edit it. Upon acceptance, the entry is consolidated into a Hazard Requirement Key (HRK) and made available for export into the existing documentation format. This stepwise workflow of the proposed chatbot is illustrated in Figure 3.4.

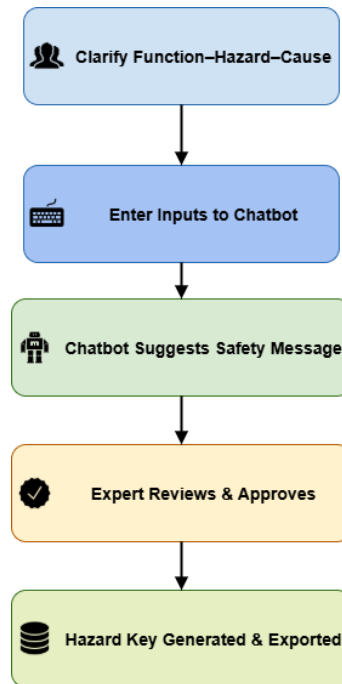


Figure 3.4: Chatbot workflow

In parallel with these workflow illustrations, a context diagram Figure 3.5 defines the system boundary of the chatbot and its placement within the regulated documentation process. At its center is the chatbot itself, which transforms structured inputs into candidate safety messages while preserving expert oversight. Surrounding it are the primary human actors. In the current prototype, domain experts in medical imaging safety and regulatory affairs provide the FHC inputs and validate the chatbot’s proposed messages.

Identity and Single Sign-On (SSO) services enforce secure authentication and authorization, while also restricting access to the FAISS database, which stores vectorized representations of Hazard Requirement Keys (HRKs) for semantic retrieval. It ensures that sensitive documentation is accessible solely to authorized users. Taxonomy and terminology services supply standardized hazard categories, ensuring consistency across drafted safety messages.

By combining human actors, enterprise systems, and governance services into a single view, the context diagram highlights both the operational scope of the prototype and the safeguards that ensure security, accountability, and regulatory alignment.

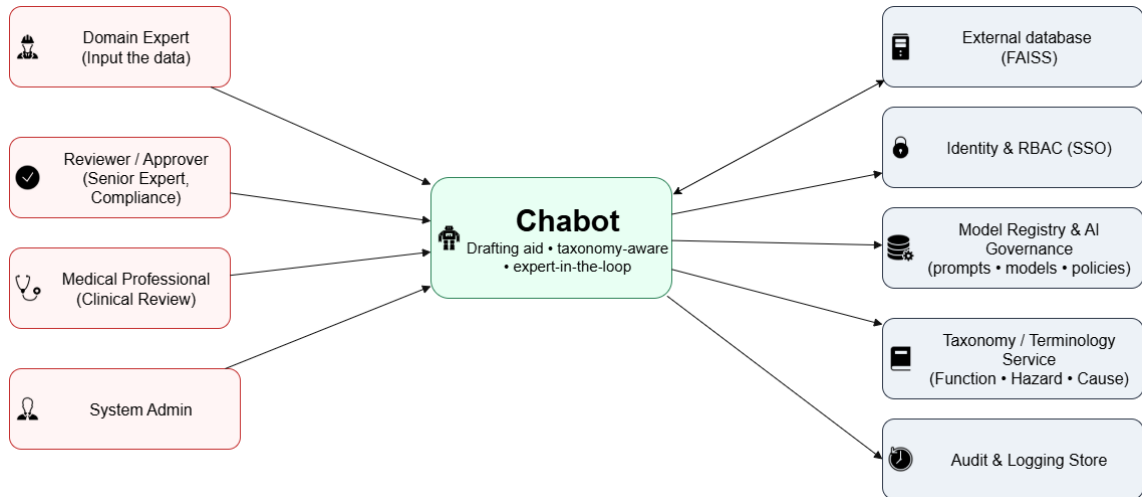


Figure 3.5: Context Diagram

3.3 Use Case Model

This part makes the use case models for the uses shown in Section 3.2. There are three parts to it. To begin, the requirements are put into formal categories and divided into functional and non-functional requirements. Then, a use case diagram shows how the expert and the chatbot will interact to meet these needs. Lastly, the use cases are explained in detail to show how the chatbot helps create, review, and generate hazard keys while still giving experts the final say.

3.3.1 Requirements

To make candidate safety messages, the chatbot needs complete input in the form of function, hazard, and cause. The User Interface (UI) has a search box with the function="", cause="", and hazard="" to ensure all of this. If the data isn't in the right format, the retrieval process won't start, and the chatbot will ask you to enter it again. The chatbot will only accept that prompt and start getting the safety message when the three inputs are in the right format.

After the chatbot retrieves or generates candidate safety messages, it shows them to the expert for review. The expert can choose one of the options and edit it if they want to. Once the safety message is confirmed, the HRK for that safety message is retrieved. At this point, the system puts the validated message together with its function, hazard, and cause to make a HRK. Each HRK is kept as a unique identifier, which makes it easy to find in safety documents and provides a consistent reference for integration with risk management systems.

The prototype also satisfies a few non-functional requirements. The given chatbot only allows input data in a structured format for all interactions, allowing the system to have consistency and maintainability. The system keeps track of internal links by giving each interaction with the chatbot a local identifier when it enters the search query. You can see the input, the retrieval step, and the safety message that came out of it in the validation logs. Keeping the log files of all the interactions allows it to audit the process. Accountability is assured by giving only domain experts the ability to validate or finalize the safety message. Only the developer is currently using the prototype, so it lacks access control systems. Despite being a research prototype, these non-functional requirements enhance the system's reliability, transparency, and expert oversight.

3.3.2 Use Case Diagram

Figure 3.6 shows the three main ways to use the prototype. In Use Case (UC)-1, the expert types out the function, hazard, and cause. Every query is saved with a locally generated identifier so that the whole interaction can be shown later in the validation logs. If the inputs are missing or not in the right format, the system sends back the same query and doesn't start the retrieval. Only inputs that are set up correctly start the retrieval and generation step, which makes one or more possible safety messages.

UC-2 is about expert validation. All retrieved or generated message options are displayed, as various formulations may correspond to the same FHC combination. The expert looks over these choices, picks the best one, or makes changes if needed. The version that the expert agrees on is the one that is used to generate the hazard key.

UC-3 The hazard key is generated. The system combines the function, hazard, cause, and the confirmed Safety Message after the message has been checked. It then gives the HRK for it. These make up the final hazard key. When the expert types “export”, the hazard key and the local identifier are added to the export and validation logs. Each entry links back to the input and safety message.

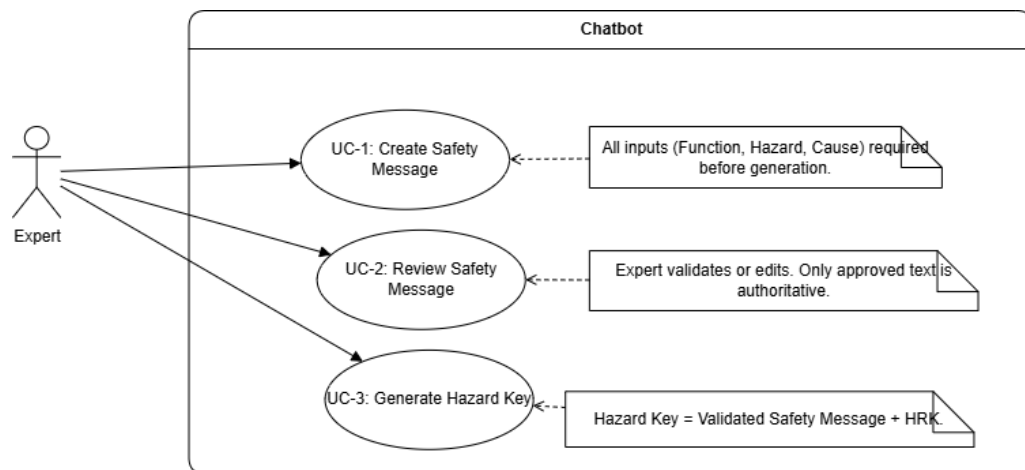


Figure 3.6: Use Case Diagram

3.3.3 Description of the Use Cases

In UC-1, they begin by typing in the function, hazard, and cause. The chatbot only works when fully structured inputs are entered; this keeps the formatting the same and doesn’t allow incomplete or wrong formats. A local identifier is attached to each entry so that later logs show the whole history of the interaction.

In UC-2, the expert’s job is to approve the right safety message. The chatbot can show a few candidates, but only the expert can say which one is correct. You can accept messages as they are or edit them if you like. The only authoritative output is the final message that has been checked by an expert.

In UC-3, the validated message and its structured inputs are put together to make the HRK. The system sends back the hazard key and connects it to its own identifier. If the expert finds mistakes at this point, they can go back to the previous steps, correct the inputs or message, and make the key again before exporting.

By breaking the workflow down into these three use cases, it is clear who is responsible for what: the chatbot speeds up retrieval and drafting, while the expert makes sure everything is correct, validated, and compliant.

3.4 Design Solutions per Use Case

Building on the use case model defined in Section 3.3, this section specifies the design solutions based on the three use cases established earlier and provides the structural basis.

Following the UCSD methodology, these use cases are expanded upon by additional models that depict the system from various viewpoints. The structure of this section is as follows: We have subsection 3.4.1 by outlining the information needed to represent functions, hazards, and causes. It then explains the sequence of interactions across the three use cases in subsection 3.4.2. We then have the 3.4.3 which shows the 3 main components of the chatbot for the given 3 use cases, and then the 3.4.4 shows us how the system will look when the user is interacting with it and then we see the architecture model 3.4.5. Lastly, this subsection explains the architecture used for implementing the hazard key generating system.

3.4.1 Information model

The information model lists the entities, properties, and relationships that the chatbot needs to be able to handle the three main use cases. The prototype's main input entities are the FHC triplets that the experts provide to the chatbot. Relationships are maintained between all entities to ensure traceability from the original input to the finalized hazard key.

In UC-1, the necessary information includes the function, hazard, and cause. All components must be fully present prior to the generation of candidate messages. If one or more elements are absent, the chatbot will not initiate the retrieval of the safety message; instead, it will present the same search message indicating formatting issues or incorrect input. Until these issues are resolved, the workflow will not retrieve the safety message, thereby interrupting the process and explicitly prompting the expert to supply the missing information. The system generates one or more candidate

safety messages solely when a complete triplet is present. At this juncture, each triplet is allocated a local identification for every interaction with the chatbot.

In UC-2, the information entities include the candidate safety messages generated by the chatbot, the expert's validation decision, and any edits made to the text. Once the safety message is selected and confirmed by the domain expert, each validation action is logged, creating a validation history that links the final approved message back to both the original FHC triplet.

In UC-3, the validated safety message is consolidated as FHC, and the HRK is incorporated into a complete hazard key. The hazard key serves as the final information entity and provides a stable reference for downstream documentation in Product Lifecycle Management (PLM), Quality Management System (QMS), and Risk Management System (RMS) systems. The resulting information model thus consists of five key entities: function, hazard, cause, safety message, and HRK, together with the relationships that link them across the three use cases.

Figure 3.7 illustrates the flow of information across the use cases, showing how inputs are transformed and enriched at each stage until they form the final hazard key.

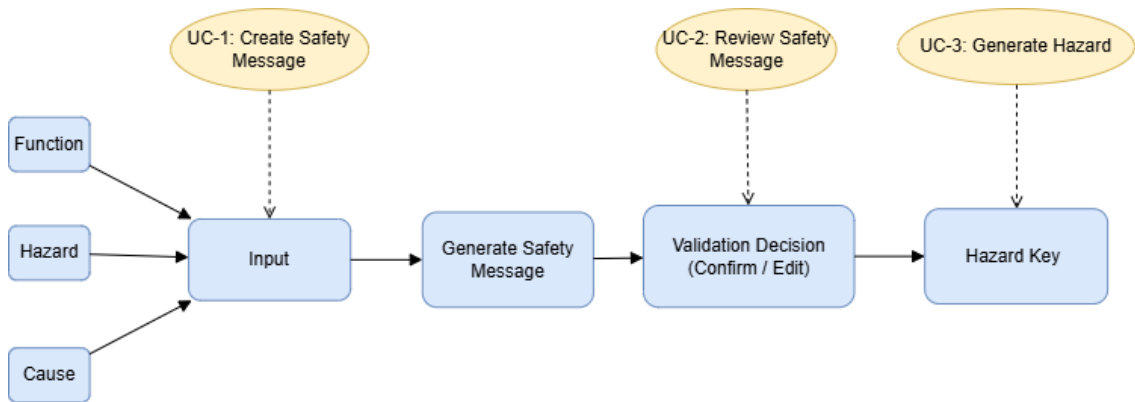


Figure 3.7: Information model

3.4.2 Sequential Model

The sequential model specifies how interactions between the expert and the chatbot unfold each time for all the given use cases.

In UC-1, the expert initiates the workflow by supplying the function, hazard, and cause. The chatbot immediately performs a completeness assessment. If the input is

not in the expected format, the system interrupts the process and prompts the expert to provide it in the correct format through the dialogue interface. Only when the input is complete does the chatbot retrieve one or more candidate safety messages, which are passed forward for review.

In UC-2, the chatbot presents the set of candidate messages to the expert. The expert can then confirm one of the options, editing if necessary. The system records all candidate messages together with the validation decision, but only the expert-approved messages are stored for reviewing later.

In UC-3, the validated safety message is combined with its original FHCTriplet into a consolidated HRK. This unique identifier is assigned to the key to ensure there is traceability across systems. Before finalization, the expert reviews the hazard key; if inconsistencies are detected, the workflow returns to UC-1 or UC-2 for correction. Only once the hazard key is validated is the export function performed. The sequential model is illustrated in Figure 3.8, the interaction between the user and the chatbot.

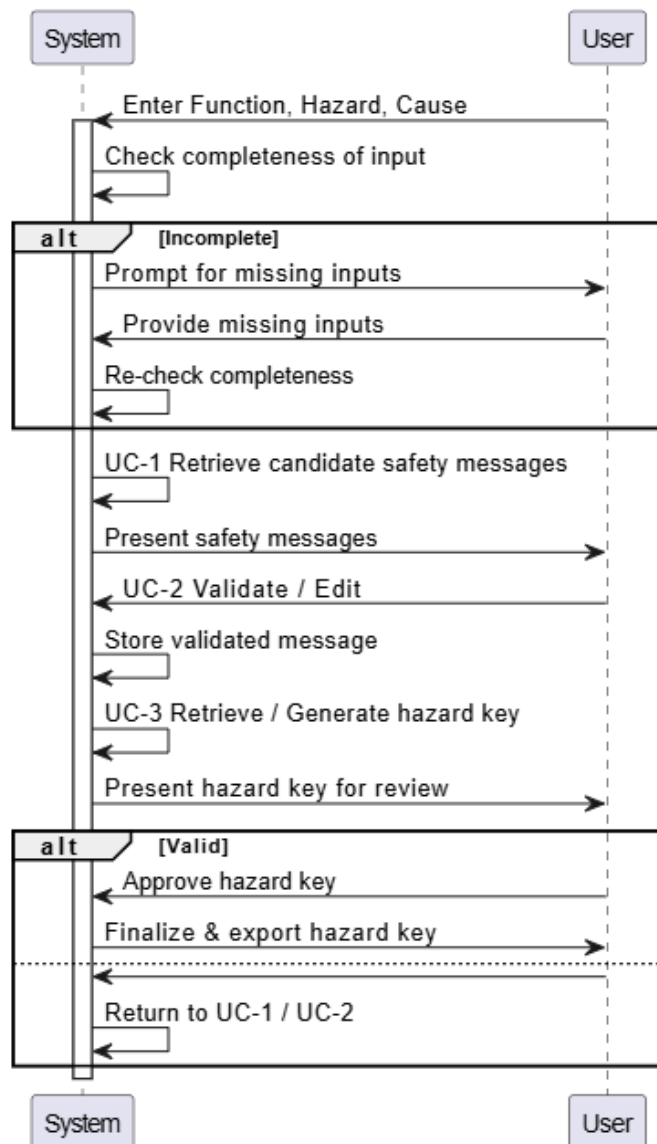


Figure 3.8: Sequential model

3.4.3 Component Model

The components model breaks down the hazard key generation into functional modules that fit with the three use cases: creating messages, validating experts, and generating and exporting hazard keys. An input parser first takes care of incoming FHC triplets. It normalizes the text and turns it into a structured form. This representation is sent to the Retrieval Component, which uses an FAISS index of existing hazard keys and safety messages to find exact or almost exact matches. If

the exact match is found (UC-1), the message is sent back right away and displayed on the UI.

If not, it is sent to the Generation Component, which revokes the local LLM and generates one or more possible safety messages following a strict JavaScript Object Notation (JSON). It then displays all the options for expert selection and editing (UC-2). After the user confirms a message, the hazard key is generated for the corresponding safety message (UC-3). Finally, an export Component waits for specific export requests and writes the current state of the hazard requirement key list and the validation log to two Excel workbooks. This makes sure that downstream safety documentation processes can use the validated outputs.

This is illustrated in Figure 3.9, showing how the system handles an FHC input in three main steps. The Input Processor first normalizes the input, and then the Exact Match Search (FAISS) checks it against existing records. If a validated message already exists, it goes straight to the Review and Confirm Module. If not, the request goes to the LLM Generator, and the Message Options Handler organizes the outputs before they are reviewed. Once the user approves a message, the Hazard Key Generator sends the result to the Export Module, which makes the final Excel output.

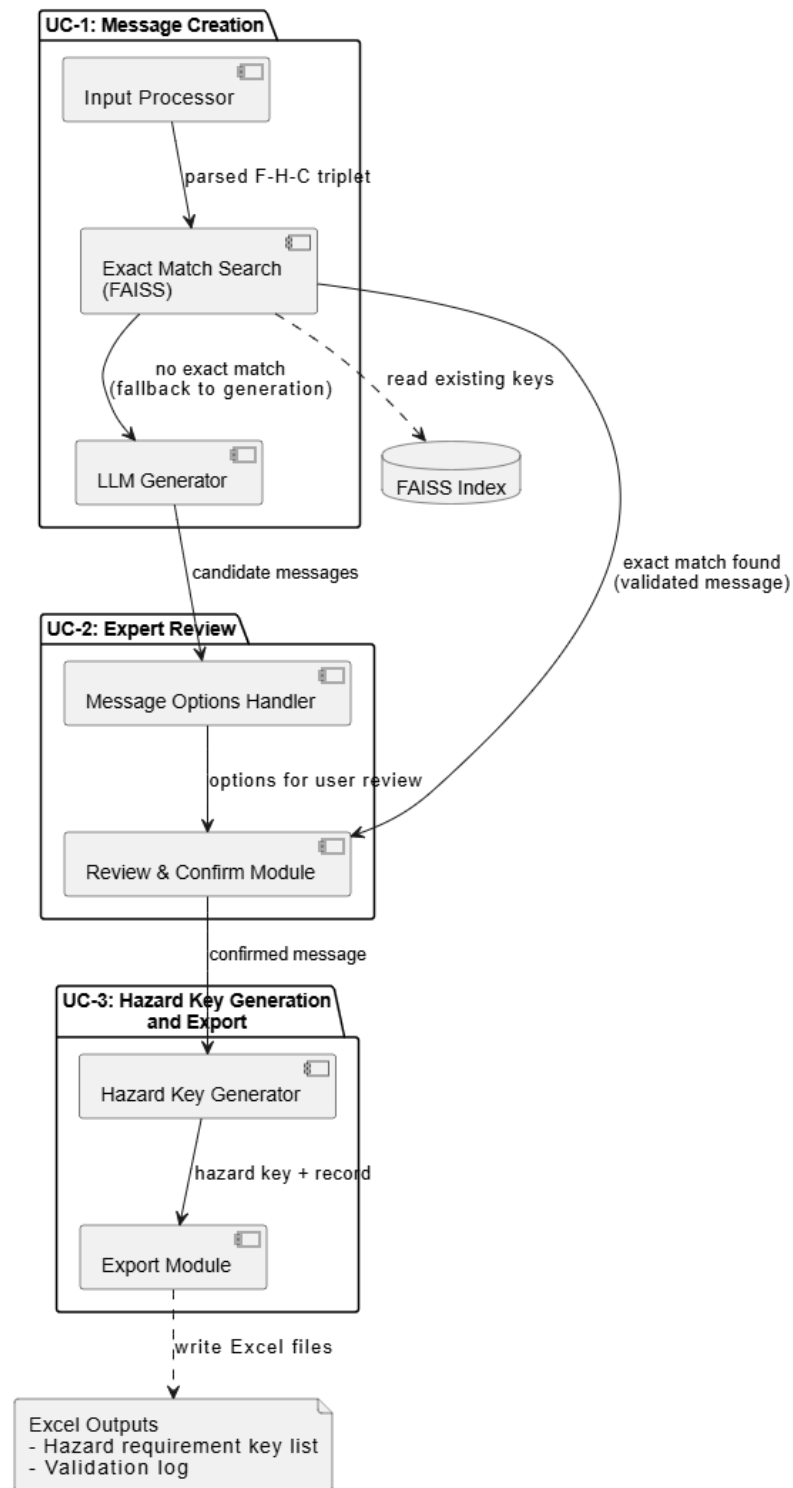


Figure 3.9: Components model

3.4.4 User Interface Model

The interface model shows how the user interacts with the system through the chatbot interface. The user and the system interact with each other in all three cases. The user types in the FHC into the input search box. If there is a match, the interface shows an exact-match safety message from the FAISS index. If there is no match, it shows a set of LLM-generated candidate messages. The interface makes it easy to choose, edit, and confirm a message during the review stage. Once the user confirms, the interface shows the generated hazard key and accepts the “export” command after the message has been approved. This step starts the process of making the hazard requirement list and validation log Excel files. The figure 3.10 shows what the user will see at each point of interaction, depending on the three use cases.

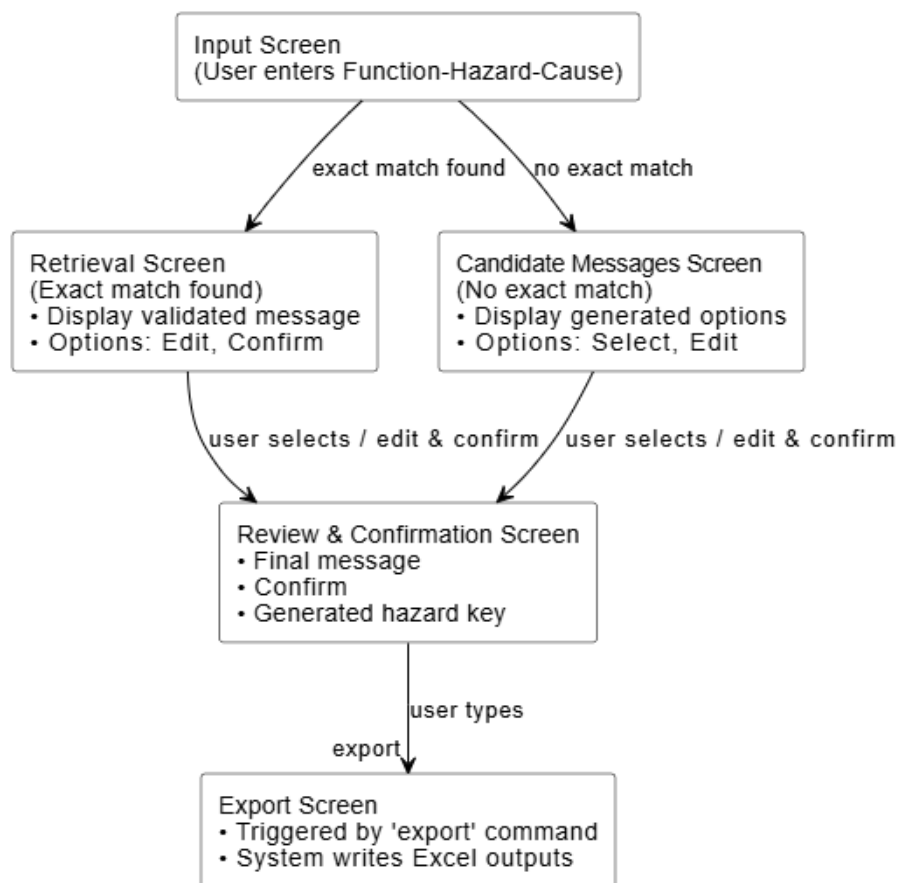


Figure 3.10: The User Interface model

3.4.5 Architecture Model

The architectural model illustrates a singular machine system where all logic operates within a Python environment on that machine. A UI application initiates on the laptop, providing a web interface on localhost. The user accesses this interface via a browser, which solely displays the user interface; the Python script does all processing tasks. For UC-1, the UI sends the FHC input to the backend code. The backend code then checks a local FAISS index for an exact match. If it doesn't find one, it calls the local LLM. In UC-2, the same UI shows the messages that were found or created and keeps track of the user's edits and confirmation. For UC-3, the backend goes back to the FAISS to obtain the hazard key for the safety message that was approved. It then writes two Excel files to the laptop's file system. This has been clearly illustrated in the figure 3.11.

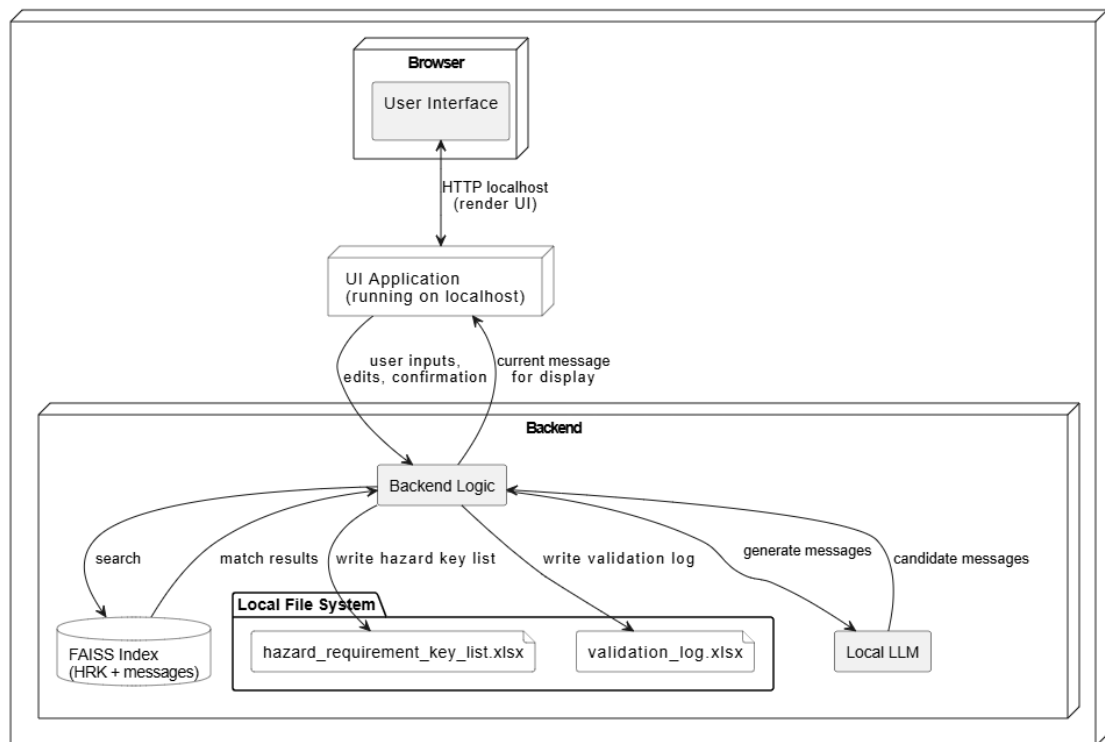


Figure 3.11: Conceptual architecture model

3.5 Summary and Discussion

This chapter laid the groundwork for the theoretical conceptual foundations for the chatbot using UCSD. We identified the three main use cases after defining who, where, and who would be involved.

In this chapter, all the models are presented on the foundational ground of these three use cases. The information model made it clear what data each use case requires and produces. The sequential model traced the step-by-step flow of each use case from input to output. The components model shows what components are needed in the implementation of the chatbot. The user interface model represented how each use case appears visually to the expert user on the user interface.

Finally, the architectural model showed how each use case is supported technically within the local backend and UI setup. Together, these models allowed each use case to be examined from multiple angles: data, behavior, computation, interaction, and infrastructure, while keeping the design consistent across representations.

This approach provided clarity on what the system must do and how each part supports the expert's tasks. It also highlighted key design choices, such as using a local prototype for controlled evaluation of LLM outputs and embedding traceability and compliance features early in the design. At the same time, the modeling revealed areas that will need more work during implementation, such as scaling retrieval to larger datasets and validating the balance between automation and expert oversight.

The chapter created a structured and traceable design where the three use cases remain the anchor across all models. This creates a solid foundation for the chapter 4, which moves from these conceptual models to the practical implementation of the system.

4 Implementation

This chapter presents the implementation of the safety chatbot prototype, translating the conceptual models. It addresses RGS1.1, which concerns the design and implementation of the chatbot, and RGS1.2, which focuses on the integration of compliance and data protection mechanisms into the overall system architecture.

Section 4.1 introduces the core technologies and frameworks that support the prototype. Section 4.2 describes the design and implementation of the chatbot itself, including the configuration of the validated FHC dataset in Section 4.2.1 and the RAG pipeline presented. Section 4.3 outlines the integration of compliance features into the system architecture. Finally, Section 4.4 provides a discussion and summary of the implementation outcomes, reflecting on the technical findings, identified challenges.

4.1 Core Technologies

The safety chatbot prototype was implemented in a controlled Python environment. The development environment was based on **Python 3.11.9**, managed within a dedicated virtual environment, and executed in **Visual Studio Code**.

At its foundation, the system employs the **Ollama 3** LLM, chosen for its accessibility and suitability for secure, offline experimentation. Ollama operates as a stand-alone local model. This design significantly reduces dependency on external infrastructure. The model is accessed through the **Ollama Python client (0.5.3)** and orchestrated via the **LangChain framework (0.3.27)**, which enables modular construction of RAG pipelines and controlled prompt execution.

The system uses **OllamaEmbeddings** with the `nomic-embed-text` encoder to turn FHC triplets into dense numerical vector representations for semantic encoding. These

embeddings are indexed and retrieved using **FAISS (1.12.0)**, a high-performance vector database developed by Facebook AI Research. The supporting ecosystem includes **NumPy (2.3.2)** and **pandas (2.3.2)** for structured data manipulation and **Streamlit (1.49.1)** for delivering an interactive user interface.

Within the safety chatbot system, Streamlit serves as the conversational front end through which domain experts input FHC triplets, review candidate safety messages, and confirm or refine final outputs. Within the safety chatbot system, Streamlit serves as the conversational front end. **Openpyxl (3.1.5)** is used to export the data to the Excel file.

4.2 Safety chatbot

Building on the core technologies outlined in Section 4.1, this section describes the implementation of the safety chatbot prototype. The implementation is done in two main stages. First, validated safety documentation was pre-processed and embedded into a FAISS vector database to enable retrieval based on semantic similarity. Second, the RAG pipeline was implemented to combine FAISS retrieval with the **Ollama 3** model for candidate message generation, user validation, and export.

Subsection 4.2.1 explains the data preparation and embedding setup, while Subsection 4.2.2 presents the integrated retrieval, generation, and validation workflow.

4.2.1 Implementation Setup and Data Preparation

The first step in building the safety chatbot was organizing the material needed for semantic retrieval. The confirmed safety messages were placed in a simple Excel sheet with five columns: Function, Hazard, Cause, Safety Message, and Hazard Key. Each row holds one complete FHC entry together with its safety wording, indexed by a unique HRK. Excel was chosen mainly because it fits smoothly into current documentation routines. Domain experts can review and update the information in a format they use every day, while the fixed FHC layout stays intact. The sample of the dataset is shown in Figure 4.1.

	A	B	C	D	E	F	G	H	I
	F	Function	HG	Hazard	CU	Cause	M	Safety Message	Hazard Key
1		▼	▼	▼	▼	▼	▼		▼
2	1	patient registration (POF)	1	incorrect diagnosis basis	3	wrong entry of patient orientation	3	Operator Manual: CAUTION Source: Wrong entry of patient position! Consequence: Wrong basis for diagnosis. Countermeasure: Make sure that the patient position is correct.	hz_ga_F01G01U03M03
3	1	patient registration (POF)	1	incorrect diagnosis basis	3	wrong entry of patient orientation	4	Operator Manual: CAUTION Source: Wrong labeling of the sides when the patient is repositioned! Consequence: Wrong basis for diagnosis and treatment. Countermeasure: Correct the patient orientation when the patient is repositioned. Review the labeling of the sides, to avoid wrong operative intervention.	hz_ga_F01G01U03M04
4	1	patient registration (POF)	1	incorrect diagnosis basis	4	wrong entry of patient sex or age (OSTEO, PULMO)	1	Operator Manual: CAUTION Source: Wrong entry of patient sex or age! Consequences: Wrong basis for diagnosis. Countermeasure: Make sure that the patient sex and age are correct.	hz_ga_F01G01U04M01
5	1	patient registration (POF)	1	incorrect diagnosis basis	6	inconsistent patient data or image information	2	Operator Manual: CAUTION Source: Inconsistent patient data or image information for syngo CT and syngo.CT postprocessing applications! Consequences: Wrong basis for diagnosis. Countermeasure: Do not modify patient data (Edit > Correct in the Browser menu) after results have been calculated.	hz_ga_F01G01U06M02

Figure 4.1: Input dataset used for embedding and retrieval.

The embedding setup was checked using the Python embedding script, which confirmed that each input text is converted into a 768-dimensional vector. This size defines the feature space used by the vector database and influences both retrieval quality and storage needs. The **OllamaEmbeddings** model `nomic-embed-text` was chosen because it produces compact semantic vectors that work well for short technical phrases and hazard-related terms. The preprocessing script handles the main data loading and embedding workflow. It imports the validated Excel file, normalizes the column names, and removes duplicated or incomplete entries to keep the dataset clean. Before generating embeddings, the script checks that every row contains all required fields; any record that fails this check is logged and skipped. This ensures that only complete and consistent data enters the retrieval index.

```

1 import pandas as pd
2 df = pd.read_excel("data/hazard_requirement_key_list.xlsx")
3 df = df.dropna(subset=["Function", "Hazard", "Cause", "Safety Message"])
4 df = df.drop_duplicates(subset=["Hazard Key"])

```

Listing 1: Code snippet showing data import and validation checks.

Listing 1 illustrates the initial data validation step, where the Excel file is imported, and all rows with missing or duplicate entries are automatically removed. These preprocessing operations ensure that only complete and unique FHC triplets proceed to the embedding phase. The use of pandas enables reproducibility and transparency in the cleaning process, as validation criteria are explicitly defined within the script.

The terminal output confirmed that 511 valid rows were loaded from the Excel dataset, all containing non-empty safety messages, and that 584 text chunks were successfully created for indexing.

Each validated record was then converted into a structured document object containing both the textual content and metadata fields. Longer entries were segmented using the `RecursiveCharacterTextSplitter`, which divides text into overlapping chunks of 700 characters with a 100-character overlap. This configuration preserves sentence boundaries and maintains semantic continuity between adjacent segments, thereby improving retrieval accuracy when multiple context fragments are used.

Each chunk was embedded and stored locally in the FAISS vector database. The resulting index is written to the `indexes/safety_faiss` directory and serves as the foundation for all subsequent retrieval operations within the chatbot. The following listing shows the essential steps of the embedding and index creation process.

```
1 # Create embeddings and build FAISS index
2 emb = OllamaEmbeddings(model=EMBED_MODEL)
3 vector_db = FAISS.from_documents(chunks, emb)
4 # Save index locally for retrieval operations
5 vector_db.save_local("indexes/safety_faiss")
6 print("Index successfully created and saved.")
```

Listing 2: FAISS index creation and local storage process.

Listing 2 demonstrates how the FAISS index is generated and persisted locally. Each processed document chunk is converted into a high-dimensional embedding vector, which is then indexed using FAISS to enable efficient semantic search. The index is saved in a designated local directory to preserve confidentiality and maintain a reproducible reference for later retrieval.

The final log output confirmed the successful embedding of 511 records and creation of 584 text chunks. The resulting FAISS index remains lightweight, requiring only a few hundred megabytes, and can be executed efficiently on a standard Central Processing Unit (CPU). All embeddings and index files are stored locally.

In summary, this stage establishes the validated, semantically indexed dataset that forms the knowledge base for all chatbot retrieval operations. The resulting index enables high-precision semantic search and provides the main foundation for the RAG pipeline.

4.2.2 Retrieval–Augmented Generation Pipeline

The safety chatbot integrates a RAG pipeline that couples semantic search with controlled text generation. This hybrid architecture ensures that responses remain grounded in validated safety messages, thereby minimizing hallucination and preserving traceability. The implementation logic mainly consists of the interaction between the retrieval layer, the generation module, the validation log, and the user interface.

When the user provides a structured query through the Streamlit interface comprising the FHC triplet, the chatbot embeds this input using the same 768-dimensional representation as the indexed dataset. The embedded query is compared against the FAISS index via the `load_retriever()` function, which returns the most semantically similar entries. If an exact match is found, the corresponding validated safety message is retrieved and displayed directly to the user, serving as the ground truth. This mechanism enforces the reuse of approved content and consistency.

The controlled generation is revoked when no exact match is found. The fallback routine `generate_candidates()` constructs a structured prompt using the FHC inputs together with semantically related references. It then invokes the locally hosted LLM llama3 via the ChatOllama interface. The model is constrained to output a strict JSON schema containing the keys level, Source, Consequences, and Countermeasure. A temperature parameter of 0.2 was selected to balance linguistic diversity with deterministic behaviour, ensuring that generated messages remain consistent and auditable. Values closer to zero favour accuracy and reproducibility, while higher values encourage more creative phrasing.

```

1 retriever = load_retriever(k=12)
2 docs = retriever.get_relevant_documents(make_query(f, h, c))
3 exact_docs = [d for d in docs if exact_match(d, f, h, c)]
4
5 if exact_docs:
6     # Direct reference reuse
7     options = [normalize_reference_text(get_ref_text(d)) for d in exact_docs]
8 else:
9     # Controlled fallback generation
10    ref_texts = [get_ref_text(d) for d in docs if get_ref_text(d)]
11    options = generate_candidates(f, h, c, ref_texts, NUM_CANDIDATES)

```

Listing 3: Core retrieval and generation logic.

Listing 3 illustrates the main control flow of the RAG pipeline. The first block queries the FAISS index for semantically similar safety statements. When validated references exist, these are normalized and reused directly. If no such references are found, the second block prepares contextual text and invokes the generation function to synthesize new, structured safety messages.

This logic allows deterministic retrieval and controlled generative flexibility to coexist within the same execution path. The retrieved or generated safety message options are displayed within the Streamlit interface. When a user submits the FHC triplet, Streamlit triggers the Python backend to execute the retrieval-generation pipeline and dynamically updates the interface with candidate messages. The framework’s session state ensures that user selections, edited text, and confirmation steps persist throughout a session. This design allows seamless front-back communication while keeping all computation and data storage local workstation. As we can see, the UI of the safety chatbot shows the retrieved safety message in Figure 4.2.

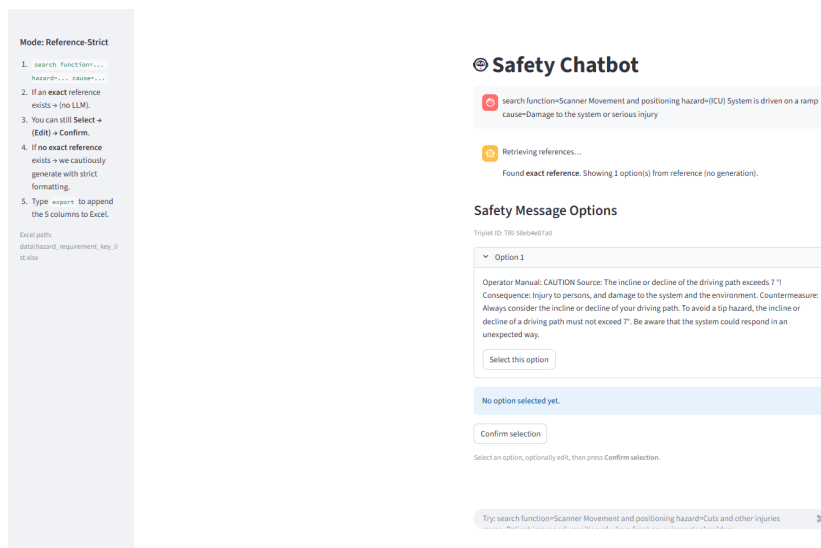


Figure 4.2: User interface displaying a retrieved safety.

Additional user interface views demonstrating the generated message options and the manual editing interface are provided in Appendix A.1 for reference. These supplementary screenshots illustrate the user validation and confirmation workflow in detail.

4.3 Compliance Architecture

In chapter 2, we discussed various concepts about governance. This chapter expands upon such concepts. In Section 2.3, we discussed the legal and regulatory frameworks for AI in healthcare. In Section 2.4, we translated these concepts into practical governance principles such as transparency, data minimization, accountability, and human oversight. The safety chatbot functions solely with anonymized hazard data and operates on a single workstation.

Regulatory Basis and Design Principles

The GDPR principle of data minimization is followed by only allowing non-personal hazard information to be stored in the system and not using any external Application Programming Interfaces (APIs) or cloud services. The EU AI ACT stresses openness and human oversight. This is shown by the fact that all safety messages that are created must be looked over and approved by a domain expert before they are exported. As of now, these safety messages are stored locally on the system in an Excel file. The Trustworthy AI framework reinforces the emphasis on traceability and technical robustness, which led to the decision to keep an append-only audit log of every retrieval, generation, and validation event. The hourglass model from 2.4.1 is separating governance requirements from implementation decisions. Governance is the upper layer that limits design and the lower technical layers.

Data Access and Local Processing

At the current prototype stage, all data processing happens on the developer's laptop. There are no external users or built-in security controls. The safety chatbot works only in a local Python environment, and the dataset, FAISS index, and validation logs are all stored as regular files on the same computer. This phase lacks encryption, access control, and permission management. The system is not yet operational beyond the developer's workstation. This setup shows that the prototype is still in the early stages of testing, where the main goal is to verify that it works and behaves as expected, not yet to enforce operational security.

The prototype shows how local processing can help meet future compliance needs, even though there are no protection mechanisms in place right now. All calculations, such as retrieval, generation, validation, and export, happen offline and don't use cloud services or external APIs. This process makes sure that the system doesn't

send data outside of the local machine, even in its early stages. This does not provide legal protection for the data, but it establishes a technical benchmark for adhering to GDPR principles, such as data minimization and avoiding unnecessary data transmission after the system’s prototype phase.

In the future, development will need to add proper access-control systems, storage protections, and structured permissions that follow company rules. Some of these are Windows New Technology File System (NTFS) restrictions, Active Directory group assignments, encrypted storage, and controlled deployment on secure devices. However, there are currently no such safeguards in place, so the prototype should only be considered an internal development artifact, not a system that is ready for compliance.

Controlled Generation and Expert Oversight

The current prototype describes the mechanisms for expert oversight conceptually; however, they remain inactive in practice, as the system is exclusively utilized by the developer during implementation. The safety chatbot encompasses the functional framework for accessing current safety messages or creating new ones via the local LLM. The oversight process, which involves domain experts reviewing, editing, and approving messages, has yet to be conducted by actual users. This is designed for future implementation, necessitating expert input before the inclusion of any generated messages in the validated hazard dataset.

At this level, only the developer engages with the system; nevertheless, the architecture is structured to ensure that the outputs of the LLM are merely suggestions rather than automated modifications. The system displays retrieved or created messages during a review phase, allowing the user to revise or edit them prior to confirmation.

Traceability and Audit Logging

The prototype, while not yet implemented for actual users, incorporates a fundamental traceability mechanism to facilitate transparent development and future compliance obligations. Every interaction with the chatbot is documented in the `validation_log.xlsx` file, which includes the FHC and the locally identifier created after each successful input search to keep all interactions accountable, any recovered matches, the generated candidate messages, the final selected option, and a

timestamp. At this stage, these logs are utilized exclusively by the developer for testing and verification, rather than for formal audit review. No encryption or access limitations are implemented at this time, as the prototype operates solely on a single development laptop and is not available to other users.

```

1 def append_validation_log(triplet_id, f, h, c, gen_candidates,
2                           selected_idx, final_msg, hrk, ref_context):
3     rows = {"timestamp": [datetime.now().isoformat(timespec="seconds")],
4             "triplet_id": [triplet_id], "function": [f],
5             "hazard": [h], "cause": [c],
6             "selected_idx": [selected_idx],
7             "final_message": [final_msg], "hazard_key": [hrk or ""],
8             "ref_context": ["\n\n---\n".join(ref_context)
9                             if ref_context else ""]}
10    # Append to data/validation_log.xlsx (append-only)

```

Listing 4: Excerpt of the validation log routine ensuring traceability.

Listing 4 shows the append-only routine used to record each interaction with the prototype. First, the function captures the FHC input and assigns it a local `triplet_id`. It then stores the retrieved candidates from the FAISS index, any generated alternatives from the LLM, and the index of the option selected by the user. The final message they are also written into the HRK log. The `ref_context` field stores the retrieved reference messages or contextual information used during generation, ensuring that both the reasoning and the output can be reviewed later. Each log entry is appended to the `validation_log.xlsx` file without modifying existing rows, which preserves the history of interaction without changing. Although this mechanism is currently used only for development testing.

Identity and Access Management.

Identity and access management are not implemented in the current prototype, as the system is used exclusively by the developer and is not accessible to other employees or organisational systems. The application does not authenticate users or enforce any form of role-based permissions. All files, the hazard dataset, FAISS index, and validation logs are stored locally with access limitations restricted to the developer's standard Windows account. In an organisational environment, user identity could be linked to Windows login credentials or integrated with Active Directory, allowing actions recorded in the validation log to be attributed to specific experts. RBAC could then restrict sensitive operations such as retraining the model, modifying the

hazard dataset, or approving newly generated messages to authorised personnel only.

Regulatory Alignment

The safety chatbot, currently an early-stage prototype operational solely on the developer's laptop, has been designed to embody the fundamental governance ideas outlined. Currently, the system does not handle personal or clinical data; hence, full regulatory compliance is unnecessary. However, several design choices already align with the expectations that would apply if the tool were later introduced into a healthcare documentation environment.

Processing that takes place only on the local systems ensures that data remains on that device, hence upholding the idea of data minimization. The retrieval-first process and the explicit validation step conform to the necessity for human control, as the system refrains from making autonomous judgments or altering documentation without verification. The append-only logging procedure reflects the traceability principle previously mentioned.

The prototype is not yet functional within the organization; hence, aspects such as access control, encryption, user authentication, and lifecycle governance are deliberately excluded from consideration.

Architecture Overview

Figure 4.3 presents a simplified view of the safety chatbot's compliance-oriented architecture. The system is designed in accordance with the Model–View–Controller (MVC) architecture, which describes the user interface, processing logic, and data storage components. The view layer, integrated inside Streamlit, serves as the browser-based interface where the user inputs the FHC triplet, reviews the generated or retrieved messages, and validates the final output. The model layer is the repository for the system's data resources. The components comprise the Excel hazard dataset, the FAISS vector index, the exported hazard key list, and the validation log. The controller may directly access each component stored on the developer's workstation.

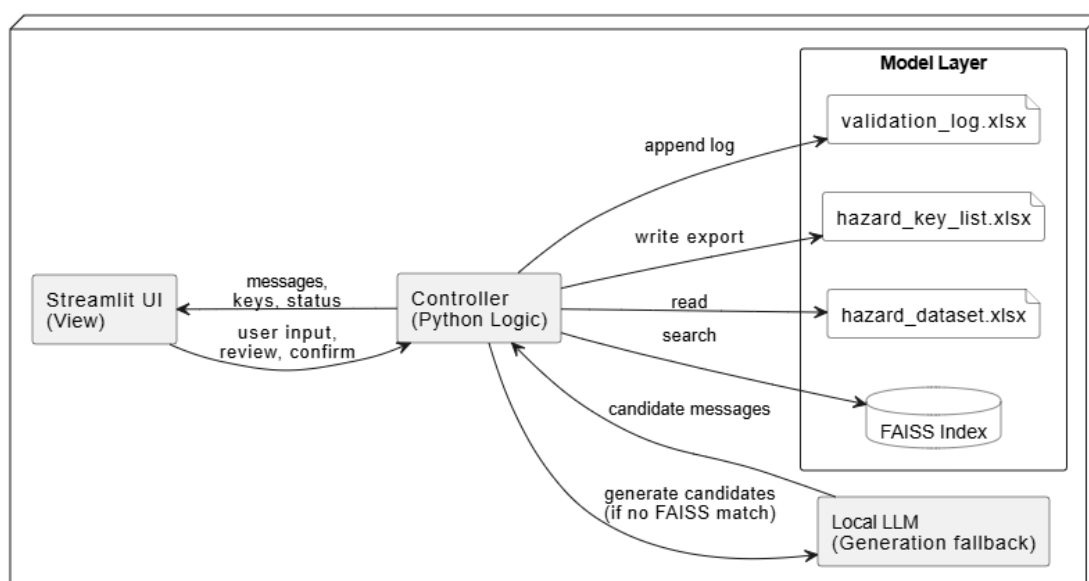


Figure 4.3: Overview of the compliance architecture.

4.4 Discussion and Summary

This chapter developed the safety chatbot prototype, converting the conceptual design from Chapter 3 into a functional RAG system. The prototype integrates a FAISS-based semantic index with a static local LLM implemented using Ollama. This architecture ensures that all outputs are verified FHC records, that the system operates deterministically, and that no external data is present in the local environment. The model does not update or learn from new inputs. Enhancements must be implemented solely through expert-sanctioned modifications to the FAISS index, allowing openness and traceability.

The design adheres to essential compliance standards, encompassing local processing, human oversight, and documentation integrity. These needs are implemented via retrieval-before-generation, structured output formats, and append-only validation logs. While access control, encryption, and multi-user authentication were not integrated at this point in the prototype phase, the architecture accommodates the future incorporation of these governance mechanisms without necessitating a redesign.

Although the implementation integrates the core components of the system, it also exposes several limitations. The research confirms that retrieval-driven approaches depend heavily on the completeness and coverage of the dataset (OQ01) (OQ02). In the current phase of the prototype, the dataset was sufficiently rich for the known inputs: the FAISS index was able to retrieve exact matches for all validated FHC combinations, and the embedding configuration based on a simple k -NN ($k=2$) setup proved adequate for this controlled scenario. However, this does not guarantee future robustness. As soon as new or previously unseen FHC inputs are introduced, retrieval performance will need to be reassessed, and any gaps in the dataset will directly affect the quality of the system's outputs. (OQ03) The research has shown that even with retrieval augmentation, hallucinations can't be eliminated. As per the early stage of the prototype, the retrieved messages are as it is in the database, no changes, but this isn't assured for when this is moved to a larger scale or newer inputs are given. This must be evaluated again for better evidence.

The safety chatbot can produce one or more options of safety messages for the same input as per the dataset and displays it for the expert to confirm it after which the hazard key is generated (OQ04) Secondly, the user verifies the completeness and formatting of inputs before retrieval or generation, partially resolving (OQ05) by preventing incomplete or improperly formatted FHC entries from entering the workflow.

This chapter illustrates the feasibility of a transparent and compliant prototype for hazard key generation. The OQs from Chapter 2 has been partially answered. The remaining questions and their implications for deployment will be analyzed in Chapter 6 as a component of future development.

5 Evaluation

This chapter assesses the implementation of the safety chatbot prototype, emphasizing its functional performance, usability, and compliance assurance within the regulated healthcare environment. The evaluation focuses on two research goals: RGE1.1, which looks at how well the chatbot generates hazard keys and retrieves safety messages, and RGE2.1, which looks at how well the system follows data protection and regulatory rules through structured compliance checks and logging.

To achieve these objectives, the evaluation employs a Cognitive Walkthrough (CW) methodology, a systematic usability framework that emulates expert interactions to assess the system's efficacy in facilitating its designated functions. Section 5.1 explains the Cognitive Walkthrough method and why it was chosen.

5.1 Cognitive Walkthrough

The CW is a practical, expert-led method used to see how well an interactive system supports the way people think as they work through a task. Wharton et al. [66] first outlined this approach, focusing on the small decisions and mental steps users make while trying to reach a goal.

Instead of relying on large user studies, the CW steps through the interaction piece by piece to spot moments where users might hesitate, become confused, or simply have to think harder than necessary. This method is mainly about checking workflow logic and how intuitive the interface feels.

For this project, the walkthrough looks at how safety engineers, risk managers, domain experts, and other specialists actually move through the prototype as they create, review, and export safety messages. It follows their thinking from the structured FHC input to the point where they generate and validate safety messages, paying

attention to how smoothly the Streamlit interface supports each step. During the walkthrough, evaluators try to mirror a user's thought process. They check whether the next action makes sense, whether the feedback is clear, and how well the system helps someone recover if they make a mistake. Doing these tasks makes it easier to see where the system's behavior lines up with what users expect and where it clearly doesn't.

The walkthrough also looks at governance needs, such as traceability, auditability, and accountability, alongside the usual usability factors. This evaluation directly supports the goals of the chapter. Section 5.2 explains how the evaluation was organized and what happens at each stage, and Section 5.3 covers what was observed, what stood out, and what challenges came up. These findings feed into the discussion in Chapter 6.

5.2 Evaluation Setup and Phases

Before running the CW, it was necessary to put down the basic conditions for how the evaluation would actually work. This chapter starts by laying out those pieces so the walkthrough has a clear frame of reference. The section begins by going through the key input parameters, which more or less define who the user is, what task they are trying to complete, and the situation in which they do it. After that, the section moves into the action sequences, the step-by-step path that the evaluators follow. Putting these elements together gives a structure that makes it easier to look at how users think their way through the prototype rather than just what they click on.

5.2.1 Input Parameters

For this walkthrough, the CW was carried out on the safety chatbot prototype using a fixed set of inputs so the evaluation stayed consistent. The role we focused on was that of a domain expert working through the Streamlit interface, which would also consider the administrator's view for checking logs and data consistency. A set of FHC triplets was chosen to test both retrieval and generation. Each triplet had its own local identifier so that everything could be traced later without any

confusion. The whole evaluation ran offline, and the system logged every interaction automatically using the built-in validation and export functions.

5.2.2 Action Sequences

The walkthrough was organised around four Action Sequences (AS) groups. Each one reflects a different part of how the prototype is used in practice, covering both the day-to-day operation and the governance duties that surround it.

- AS1 – Application Initialization and System Launch: Here, the user starts the Python application, which brings up the Streamlit interface and loads the local model, the FAISS index, and the logging tools.
- AS2 – Safety Message Generation Workflow: The user adds the structured FHC input. The system checks if anything is missing, looks for a matching entry in the database, and if it finds nothing suitable, asks the *Ollama* model to draft up to two message candidates.
- AS3 – Message Validation and Hazard Key Creation: The user goes through the suggested messages, edits the preferred one if needed, and confirms the choice. At this point, the system retrieves an existing hazard key (HRK) and stores the full record for traceability.
- AS4 – Administrative Oversight and Data Governance: An administrator reviews the logs, checks the exported files, and makes sure the stored records follow the expected governance rules.

These four sequences more or less capture the whole flow of how someone interacts with the chatbot, from starting it up to validating the final message and checking the system’s behaviour. The analysis is as follows.

5.2.3 Examination of the Sequence of Actions

Each Action Sequence was reviewed step by step. The goal was to understand how users think their way through the workflow, how clear the interface feels at each point, and whether the system keeps a reliable trail of what happened.

AS1 – Application Initialization and System Launch

- AS1.1 – Activate the virtual environment.
- AS1.2 – Run the indexing script.
- AS1.3 – Load the embedding model.
- AS1.4 – Launch the Streamlit interface.
- AS1.5 – Wait for the system to finish loading.

AS2 – Safety Message Generation Workflow

- AS2.1 – Enter the structured input.
- AS2.2 – System checks the input and starts the retrieval.
- AS2.3 – Query the database.
- AS2.4 – If there is no match, the LLM drafts a message.
- AS2.5 – Show the message for review.

AS3 – Message Validation and Hazard Key Creation

- AS3.1 – Present retrieved messages.
- AS3.2 – User reviews the options.
- AS3.3 – Select and optionally edit the message.
- AS3.4 – Confirm the decision.
- AS3.5 – Assign or generate the hazard key.
- AS3.6 – Prepare the message for export.

AS4 – Administrative Oversight and Data Governance

- AS4.1 – Trigger the export action.
- AS4.2 – Update the dataset.
- AS4.3 – Append the new record to the validation log.
- AS4.4 – Handle file-locking requirements.
- AS4.5 – Administrator reviews and archives the output.

5.3 Results and Analysis

This section presents the results of the CW conducted on the safety chatbot prototype. The walkthrough evaluates how effectively the implemented system supports user cognition, task completion, and compliance traceability across the defined operational workflows. Each AS represents a vital phase of system interaction, ranging from initialization to administrative oversight, and is evaluated according to the procedural steps and the associated system behavior.

Each of the above action sequences corresponds directly to the implementation logic described in Chapter 4. The initialization steps AS1 are in line with the technical setup and architecture described in Section 4.2.2. The safety message generation workflow AS2 is based on the retrieval and generation mechanisms described in Section 4.2.2 and Section 4.3. The validation and hazard key creation phase AS3 connects to the logging and traceability functions discussed in Section 4.3, ensuring transparency and auditability. Finally, the administrative oversight process AS4 relates to the governance and data management strategies presented in Section 4.3.

The screenshots and detailed visual documentation corresponding to each action sequence are provided in the Appendix—Cognitive Walkthrough; see section A.2, where each step is illustrated to support the analysis of system behavior and user interaction. Together, these sequences demonstrate how the safety chatbot prototype functions.

5.3.1 Logging Critical Information

AS1 - Application Initialization and System Launch. The process is initiated through command-line execution. However, this method could be automated using startup scripts to reduce manual work and ensure consistent configuration across sessions. It was further recommended that user documentation, which outlines installation steps, Python version requirements, library dependencies, and directory structure, be included in Appendix A.3. This method would enable future users to replicate the environment without additional technical guidance.

AS2 - Safety Message Generation Workflow. The input fields are strictly structured, requiring users to enter the function, hazard, and cause in predefined formats.

Ideally, the chatbot should allow more flexible user input, such as continuous or conversational text, while still interpreting and mapping it to the appropriate safety message. When an exact match is found in the dataset, the chatbot presently retrieves the corresponding validated safety message. In future versions, the system should also be capable of generating refined or context-specific safety messages even for existing matches. The current prototype retrieves the safety message with a caution or warning, as it is, without applying any underlying logic. There is currently no implemented mechanism to determine which signal word should be used based on the severity of harm or probability of occurrence. In the following iterations, this limitation should be addressed through a defined rule set or logic framework that automatically selects the appropriate signal word according to regulatory safety communication standards. A verification step should also be introduced to prove the signal word assignment. Each safety message currently includes a source reference, and it must also be ensured that each source concludes with an examination mark.

AS3 – Message Validation and Hazard Key Creation. The prototype at this phase only has existing hazard keys retrieved, while the ability to generate new keys for unseen inputs following the existing taxonomy is planned for future development.

AS4 — Administrative Oversight and Data Governance. In the current prototype, the system operates in an isolated environment and is not yet integrated into any enterprise infrastructure. It is expected to be deployed within a strict enterprise framework with a controlled rollout process. Given its AI components, such integration will require extensive testing and validation before organization-level use. At present, log files record all user interactions, but there are no access restrictions. The future implementations must introduce a role-based access model so that each user can only view their interaction logs. Administrative logs should also be restricted to authorized personnel.

5.4 Discussion and Summary

This chapter presented the evaluation of the safety chatbot prototype through a structured CW. Four AS were defined to represent the main operational steps of the chatbot, from initialization to administrative control. This evaluation identified the good and bad aspects of the current implementation that must be fixed before it

can be used in a business setting. During the walkthrough, the evaluators confirmed that the chatbot performs its intended functions reliably.

AS1 focused on system initialization and data loading; the prototype successfully accessed the required test data located in the `data/` directory. The process was considered stable and repeatable, but it was noted that the following versions should include automated startup scripts.

AS2 examined the generation of safety messages. The chatbot was able to retrieve validated messages based on the structured input. However, evaluators noted that the current input design is too rigid and should allow more natural and flexible interaction, reflecting the conversational nature of chatbots. Furthermore, the signal work, when used as a placeholder, must have logic in place, and the source definition of the safety message must always end with an examination.

AS3 covered message validation and hazard key creation; the system correctly retrieved existing hazard keys linked to validated messages. The user's edits maintained the integrity of the validated data by not altering the original database. At this stage, no new hazard keys are generated. One of the enhancements for this AS is that the chatbot should be extended to automatically create new keys for unseen inputs while adhering to the established taxonomy and coding conventions.

AS4 addressed administrative oversight and data governance. The current prototype operates in isolation and has not yet been integrated into an enterprise environment. Since all interaction logs are currently stored without access restrictions, a role-based access model should be implemented in future versions to ensure that users can view only their logs and that administrative data remains confidential. The process for adding new data to the database must be described in the user documentation, including validation and approval steps.

The cognitive walkthrough as a whole demonstrated that the prototype functions as intended and meets the design objectives defined earlier in the thesis. The evaluation confirmed the system's ability to process structured inputs, retrieve validated content, and maintain data integrity throughout the workflow.

In conclusion, the evaluation verified the technical soundness and conceptual validity of the safety chatbot while outlining concrete areas for enhancement. The feedback from the cognitive walkthrough is a good starting point for the next phase of

development, which will focus on operational scalability and compliance integration instead of functional validation. The insights gained here thus bridge the transition from prototype testing to enterprise-level deployment within a regulated healthcare environment.

6 Conclusion

This thesis presents the concept, implementation, and evaluation of a safety chatbot. The proposed system aims to assist domain experts in hazard key documentation. This system is a combination of the RAG pipeline, which does the semantic retrieval, and a controlled text-generation LLM.

6.1 Addressing the Research Questions

RQ1: How can a GenAI system be designed to build a user-friendly chatbot for hazard key generation in safety documentation?

The first research question was addressed by the design implementation of the chatbot. The core technique is the RAG pipeline, along with the locally hosted LLM. The system combined a semantic search layer with a controlled text-generating component that ensures that all safety messages are retrieved from a predefined secure database or, if needed, generated by the LLM under strict constraints. The above procedure is performed only when an exact match is missing. All the existing hazard key data is semantically encoded and stored in a FAISS vector database. The prototype had rich enough data for the retrieval process with the existing inputs; the vector database was not updated, and no changes were made during this phase of the prototype.

The user types the FHC input into the search box. Each input is tracked by generating a local identifier to keep the interaction accountable. When the right format is entered, the system starts retrieving the data. It will take a few seconds to a minute, and then if the exact match is found, the corresponding safety message options are displayed on the UI. When there is no exact match, the LLM generates the safety messages under strict constraints.

This workflow is implemented within a Streamlit interface, which uses the browser to host this application. The UI is designed in such a way that the domain experts can enter the input data, review, and select the safe message; it also gives the option to edit the selected messages. Once the user is confident with the safety message, we can confirm the selection by clicking on the confirm button displayed on the UI; with that, the hazard key is generated and displayed on the screen for the corresponding safety message. All this data can be exported to two different Excel sheets by typing “export” in the search box. This system is able to achieve a certain amount of consistency and compliance while remaining user-friendly, as seen in the CW.

RQ2: How can GenAI be effectively used while ensuring data protection and compliance with regulatory requirements?

The prototype runs entirely in a laptop environment, ensuring that data, whether it is the inputs or the safety message, is exactly where it is supposed to be. This setup facilitates all processing steps, including semantic retrieval and message generation. There is no use of external APIs or cloud networks. These external model calls reduce the risk of unintentional data exposure. The input data is structured in an FHC format, and there is no use of personal data, keeping the data footprint minimal with a clear and open purpose and avoiding the unnecessary processing of additional information.

Each time the user inputs the data into the chatbot, if it’s in the right format and if retrieval has started, a local identifier will be created for each interaction, which is stored in the validation log along with the timestamp. This allows some traceability and enables reviewers to check the safety messages produced and verify them. Explainability can’t be fully provided, as many things are still unknown or yet to be clarified, but this logging mechanism gives a transparent view of the retrieval and generation path. Regulatory considerations from the GDPR and the EU AI ACT were not fully implemented as a compliance feature but translated into practical safeguards appropriate for an early-stage research prototype. This feature includes data minimization, offline processing, a secure database, and traceable auditing through the log file. The evaluation showed that domain experts were able to understand how the system arrived at its outputs, which supports both transparency and accountability.

6.2 Future Research and Work

While the prototype demonstrates the feasibility of the safety chatbot, further research is required to scale, automate, and integrate the system into enterprise environments. The following directions outline potential future developments and enhancements.

Enterprise Integration and Role-Based Access

The safety chatbot needs to add RBAC and enterprise-level encryption to its system. It should also have clear role-based authentication for different people, such as administrators, risk managers, and domain experts. This way, we can decide who has access to what level of usage, like validation logs, search history, and the database. By introducing enterprise authentication and encryption, the chatbot would be able to fit in with the company's security and governance policies. This feature would enhance visibility and ensure compliance with regulations

Automation and Online Deployment

Automation of the backend processes and deployment onto a secure online platform would be the next steps for the chatbot. Currently, the developer manually handles data ingestion, index building, and validation for this prototype. We could enable continuous dataset updates and reduce the administrative workload by automating these processes through a scheduled pipeline. This feature is also important for the chatbot because it would allow newly validated safety messages, released by the risk management process, to be automatically incorporated into the input dataset as updates by minimizing manual effort and keeping the dataset up to date.

Adaptive Hazard Key and Signal Word Logic

This would be a key area for future improvement, which would be to enable the system to produce hazard keys dynamically and automatically modify the signal word. Connecting the chatbot to the Azure DevOps Global RA documentation system would also allow it to look up and assign the next available hazard key without any manual steps. Adding the signal-word decision logic would provide the chatbot the ability to choose between "caution" and "warning" based on how severe the hazard is. At the same time, users should still have the option to override this by entering their keys or signal words when creating a new safety message.

Linguistic Consistency and Reuse of Validated Text Blocks

One thing to consider later is how to make the chatbot's phrasing more consistent. Currently, it tends to write from a restricted format, but it would probably sound more reliable if it could pull ideas or wording from messages that have already been checked and approved. Those could be used almost like templates or starting points for full sentences. We already have parent terms and reusable text blocks, so those could feed into the training as well. That way, the chatbot follows the terminology and writing style we actually use in practice, and the messages produced would be clearer and more predictable

Enhanced User Interaction and Continuous Governance

The next version should include a more interactive and flexible interface, one that can handle both structured inputs and free-text entries and that lets users refine their messages step by step. It would also be useful to add a governance dashboard that shows where the data comes from, how the system is being used, and when the model has been updated. Having that visibility built in would make ongoing oversight easier and help keep the system audit-ready.

A Appendix

A.1 User Interface Views

This section presents additional interface screenshots that illustrate the user interaction and validation workflow of the Safety Chatbot.

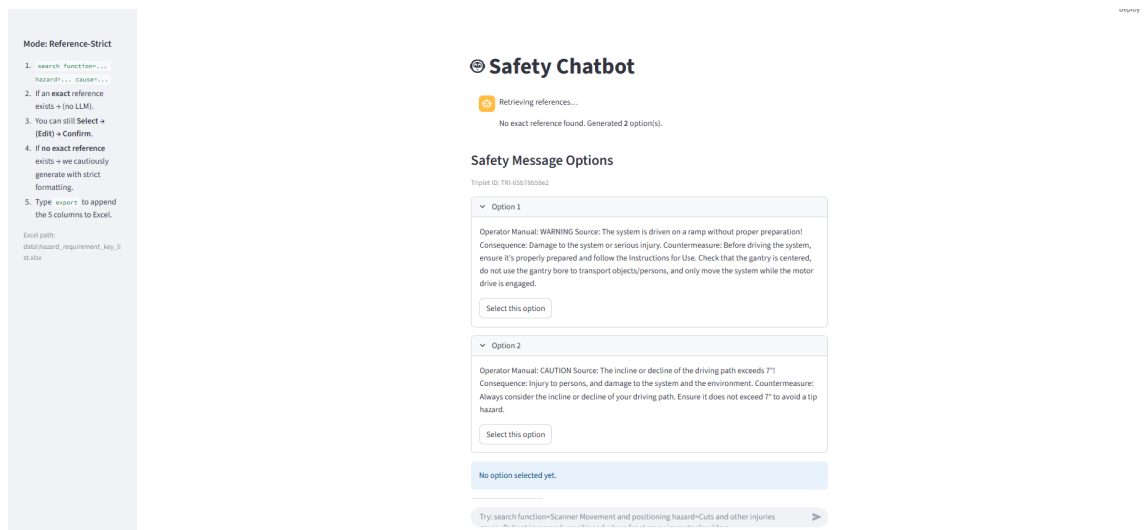


Figure A.1: Interface displaying generated safety message options when no exact match exists.

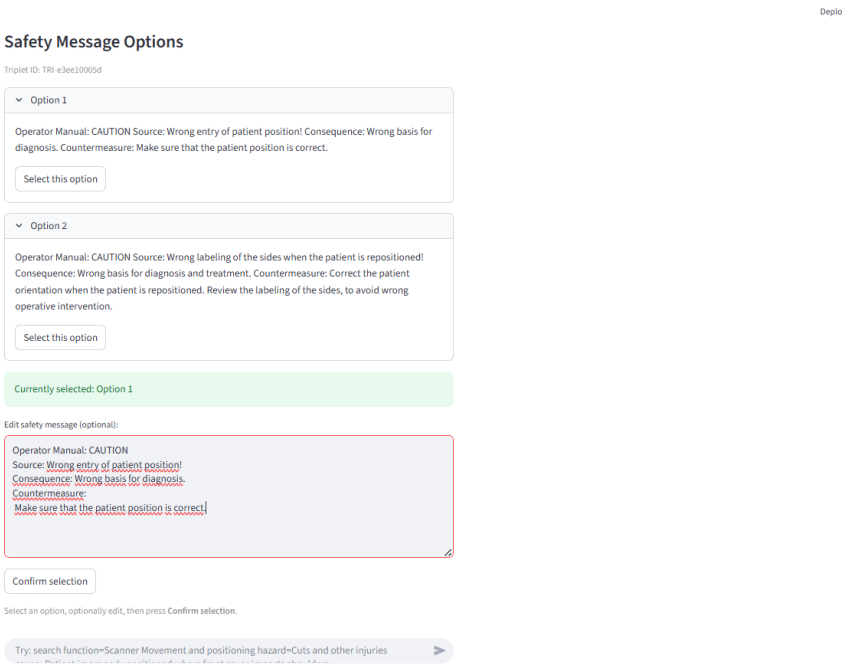


Figure A.2: Manual editing and confirmation interface for refining generated safety messages before final export.

A.2 Cognitive Walkthrough

This section presents screenshots for all the action sequences from the cognitive walkthrough section 5.3.

A.2.1 Application Initialization and System Launch

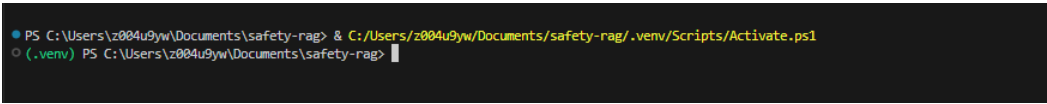


Figure A.3: Activation of the virtual environment.


```
(.venv) PS C:\Users\z004u9yw\Documents\safety-rag> python .\build_index.py
Reading: C:\Users\z004u9yw\Documents\safety-rag\data\test.xlsx (sheet=0)
Raw columns from Excel: ['F', 'Function', 'HG', 'Hazard', 'CU', 'Cause', 'M', 'Safety Message ', 'Hazard Key']
Normalized columns : ['F', 'Function', 'HG', 'Hazard', 'CU', 'Cause', 'M', 'Safety Message', 'Hazard Key']
Loaded rows: 511
Documents created: 511 (kept 511 non-empty rows)
Rows with empty 'Safety Message': 0
Chunks created: 584
```

Figure A.4: Execution of the indexing script (build_index.py).

```
(.venv) PS C:\Users\z004u9yw\Documents\safety-rag> python .\test_embeddings.py
>> from langchain_community.embeddings import OllamaEmbeddings
You can use the langchain cli to **automatically** upgrade many imports. Please see documentation here <https://python.langchain.com/docs/versions/v0_2/>
from langchain.embeddings import OllamaEmbeddings
C:\Users\z004u9yw\Documents\safety-rag\test_embeddings.py:4: LangChainDeprecationWarning: The class 'OllamaEmbeddings' was deprecated in LangChain 0.3.1 and
d will be removed in 1.0.0. An updated version of the class exists in the :class:`~langchain_ollama` package and should be used instead. To use it run `pip
install -U :class:`~langchain_ollama` and import as `from :class:`~langchain_ollama import OllamaEmbeddings`.
  emb = OllamaEmbeddings(model="nomic-embed-text")
Vector dimension: 768
First 10 values: [1.1115447282791138, 0.35601916909217834, -4.002096176147461, -0.5333396792411804, 1.2502086162567139, 0.9038958549499512, 1.2651162147521
973, 0.8111757040023884, 0.18401706218719482, 0.051854029297828674]
(.venv) PS C:\Users\z004u9yw\Documents\safety-rag>
(.venv) PS C:\Users\z004u9yw\Documents\safety-rag>
```

Figure A.5: Embedding model initialization and vector dimension output.

```
(.venv) PS C:\Users\z004u9yw\Documents\safety-rag> streamlit run matchref_chat_app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.0.209:8501
```

Figure A.6: Streamlit application launched successfully.



Figure A.7: User interface loaded and system ready for interaction.

A.2.2 Safety Message Generation Workflow

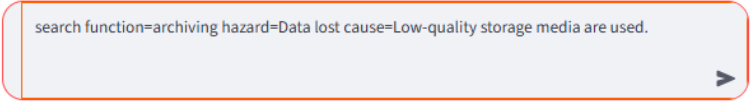


Figure A.8: Structured Input Entry.

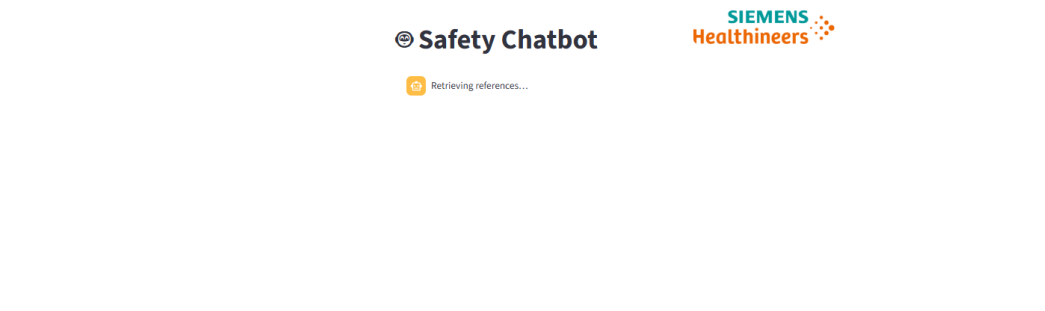


Figure A.9: Retrieval Initialization/Database Query Execution.

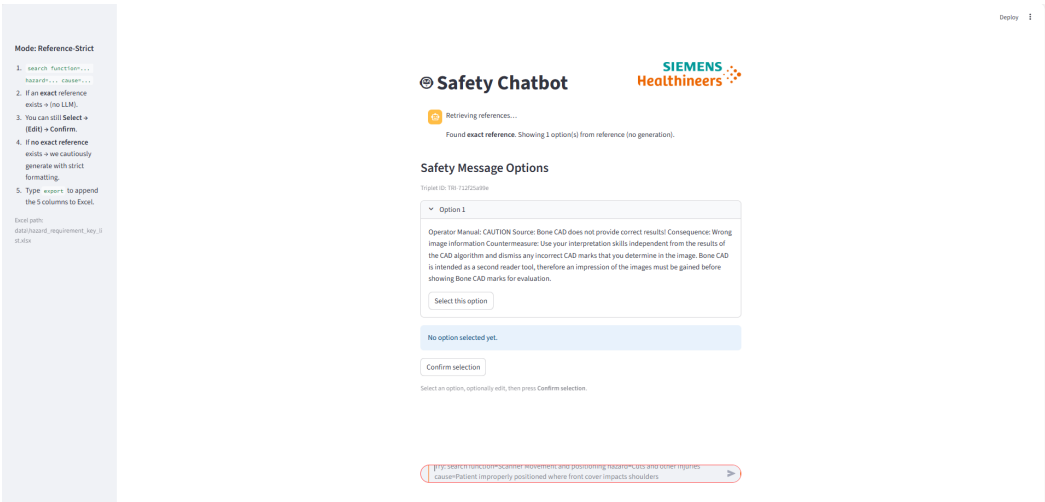


Figure A.10: Exact Reference Retrieval

 Retrieving references...

No exact reference found. Generated 2 option(s).

Safety Message Options

Triplet ID: TRI-6263d21614

▼ Option 1

Operator Manual: WARNING Source: Changes of patient information on other systems will not automatically change the patient information in syngo! Consequence: Data retrieved from PACS or other nodes seems to be misaligned or lost, but is stored elsewhere in the database. Countermeasure: Always delete the local patient data after you have successfully archived them.

Select this option

▼ Option 2

Operator Manual: CAUTION Source: Imported data may not match the existing patient information on the local system. Consequence: Data inconsistencies may occur, leading to misaligned or lost images. Countermeasure: Use other criteria from the study or series to search for the imported data, or filter the data.

Select this option

No option selected yet.

Confirm selection

Select an option, optionally edit, then press Confirm selection.

Figure A.11: No exact match, the LLM generates a message.

A.2.3 Message Validation and Hazard Key Creation

 Retrieving references...

Found **exact reference**. Showing 2 option(s) from reference (no generation).

Safety Message Options

Triplet ID: TRI-bded844637

▼ Option 1

Operator Manual: CAUTION Source: Automatically generated results, archived results, or structures used for treatment planning have not been verified by a physician! Consequence: Potentially incorrect diagnosis and treatment. Countermeasure: Before diagnosis, always make sure that automatically generated results and archived results are verified by a physician. Before Treatment Planning, always make sure that structures are verified by a physician.

Select this option

▼ Option 2

Operator Manual: CAUTION Source: Check mark for automatic archiving is configured by the system owner, for example, Auto send or Auto recon! Consequence: Wrong diagnosis Countermeasure: Always check that automatically generated results are verified by a physician or technician before diagnosis. Make sure that the results of the currently active workflow are verified. Make sure that automatically stored data on the remote system is being verified in a timely manner.

Select this option

No option selected yet.

Confirm selection

Select an option, optionally edit, then press Confirm selection.

Figure A.12: Display of Retrieved Messages/ Review of Message Options.

Widget ID: TRI_bded944637

▼ Option 1

Operator Manual: CAUTION Source: Automatically generated results, archived results, or structures used for treatment planning have not been verified by a physician! Consequence: Potentially incorrect diagnosis and treatment. Countermeasure: Before diagnosis, always make sure that automatically generated results and archived results are verified by a physician. Before Treatment Planning, always make sure that structures are verified by a physician.

Select this option

▼ Option 2

Operator Manual: CAUTION Source: Check mark for automatic archiving is configured by the system owner, for example, Auto send or Auto recon! Consequence: Wrong diagnosis Countermeasure: Always check that automatically generated results are verified by a physician or technician before diagnosis. Make sure that the results of the currently active workflow are verified. Make sure that automatically stored data on the remote system is being verified in a timely manner.

Select this option

Currently selected: Option 2

Edit safety message (optional):

Operator Manual: CAUTION

Source: Check mark for automatic archiving is configured by the system owner, for example, Auto send or Auto recon!

Consequence: Wrong diagnosis

Countermeasure:

Always check that automatically generated results are verified by a physician or technician before diagnosis. Make sure that the results of the currently active workflow are verified. Make sure that automatically stored data on the remote system is being verified in a timely manner.

Confirm selection

Figure A.13: Message Selection and Optional Editing.

Option 2

Operator Manual: CAUTION Source: Check mark for automatic archiving is configured by the system owner, for example, Auto send or Auto recon! Consequence: Wrong diagnosis Countermeasure: Always check that automatically generated results are verified by a physician or technician before diagnosis. Make sure that the results of the currently active workflow are verified. Make sure that automatically stored data on the remote system is being verified in a timely manner.

Select this option

Currently selected: Option 2

Edit safety message (optional):

Operator Manual: CAUTION
Source: Check mark for automatic archiving is configured by the system owner, for example, Auto send or Auto recon!
Consequence: Wrong diagnosis
Countermeasure:
Always check that automatically generated results are verified by a physician or technician before diagnosis. Make sure that the results of the currently active workflow are verified. Make sure that automatically stored data on the remote system is being verified in a timely manner.

Confirm selection

☒

Selected

Operator Manual: CAUTION Source: Check mark for automatic archiving is configured by the system owner, for example, Auto send or Auto recon! Consequence: Wrong diagnosis Countermeasure: Always check that automatically generated results are verified by a physician or technician before diagnosis. Make sure that the results of the currently active workflow are verified. Make sure that automatically stored data on the remote system is being verified in a timely manner.

Hazard Key: {hz_ga_F0B93U11M12}

Type exports to save 5 columns (Function, Hazard, Cause, SafetyMessage, HazardKey).

Fry: clearly identify all other involved persons and positioning; hazardous and/or other injuries
cause=Patient improperly positioned where front cover impacts shoulders

Figure A.14: Confirmation of Selection /Hazard Key Assignment

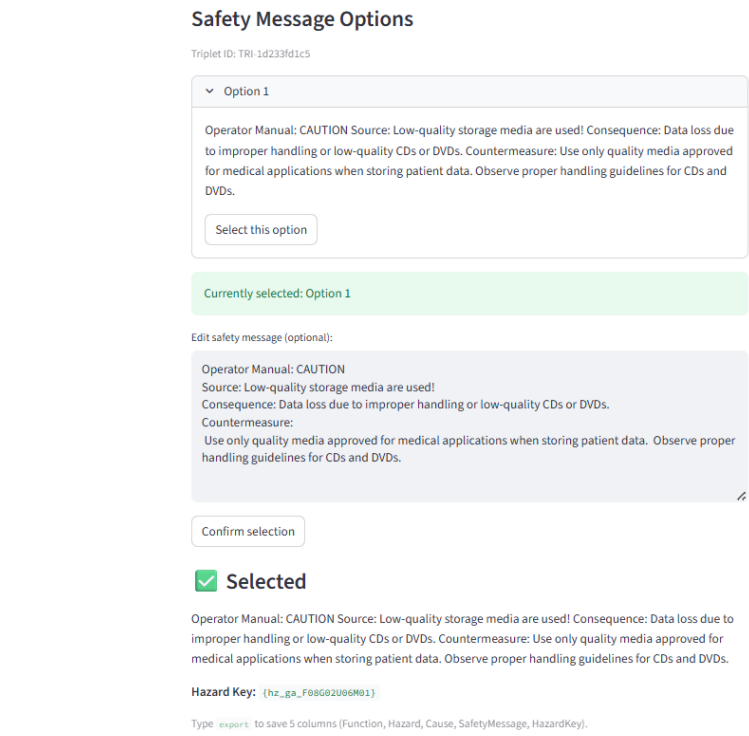


Figure A.15: Final Message Confirmation and Export Readiness.

A.2.4 Administrative Oversight and Data Governance

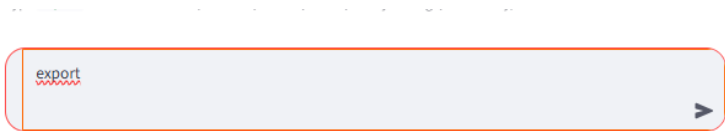


Figure A.16: Export Trigger.

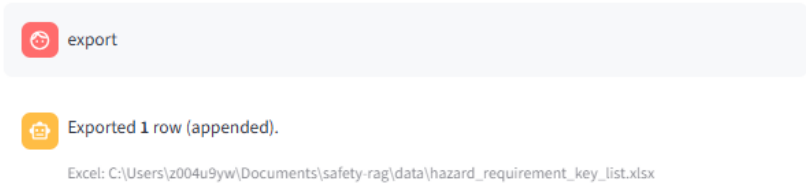


Figure A.17: Dataset Update.

1	Function	Hazard	Cause	SafetyMessage	HazardKey
2	outside influences / environment	heart arrhythmia	summation of patient leak	Operator Manual: WARNINGSource: f {hz_ga_F14G09U01M01}	
3	service hazard	Damage to the CT system	Tablet not in the cradle dur	Operator Manual: CAUTIONSource: T {hz_ga_F13G13U05M01}	
4	outside influences / environment	heart arrhythmia	summation of patient leak	Operator Manual: WARNINGSource: f {hz_ga_F14G09U01M01}	
5	outside influences / environment	damage to people and property	fire around the system	Operator Manual: WARNINGSource: f {hz_ga_F14G03U01M05}	
6	archiving	Data lost	Low-quality storage media	Operator Manual: CAUTIONSource: L {hz_ga_F08G02U06M01}	
7	Scanner Movement and positioning	(ICU) System is driven on a ram	Damage to the system or se	Operator Manual: WARNINGSource: f {hz_ga_F25G01U14M02}	
8	patient registration (POF)	incorrect diagnosis basis	wrong entry of patient orie	Operator Manual: CAUTIONSource: V {hz_ga_F01G01U03M03}	
9	archiving	Data lost	Low-quality storage media	Operator Manual: CAUTIONSource: L {hz_ga_F08G02U06M01}	
10					

Figure A.18: Validation Log Append.

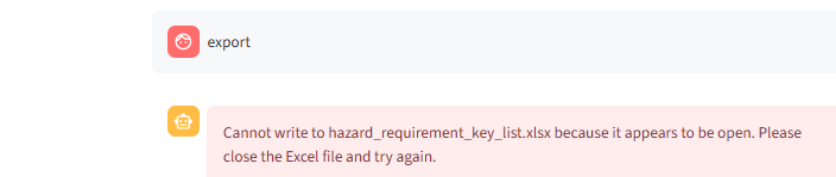


Figure A.19: File-Lock Handling.

1	timestamp	B	C	D	H	G						L	M	N	O	P
2		triplet_id	function	hazard	cause	generated_1	generated_2	selected_idx	final_message	hazard_key	ref_context					
2	2025-09-29T10:59:26	TRI-bde8b	post-procc wrong dia	Wrong dia	Operator Manual: i Operator Man			1	Operator Manual: CAUTIC {hz_ga_F06G01U118M	Operator Manual: CAUTIONSource: Automatically generated results, archive						
3	2025-09-29T10:59:29	TRI-bde8b	post-procc wrong dia	Wrong dia	Operator Manual: i Operator Man			1	Operator Manual: CAUTIC {hz_ga_F06G01U118M	Operator Manual: CAUTIONSource: Automatically generated results, archive						
4	2025-09-29T10:59:52	TRI-712f2	post-procc wrong dia	M Box: CT	Operator Manual: CAUTIONSourc			0	Operator Manual: CAUTIC {hz_ga_F06G01U99M0	Operator Manual: CAUTIONSource: Bone CAD does not provide correct resu						
5	2025-09-29T11:13:19	TRI-d4d4d	service ha: Damage tc	Tablet not	Operator Manual: i Operator Man			1	Operator Manual: CAUTIC {hz_ga_F13G13U05M0	Operator Manual: CAUTIONSource: Tablet not in the cradle during updates						
6	2025-09-29T14:31:43	TRI-712f2	post-procc wrong dia	M Box: CT	Operator Manual: CAUTIONSourc			0	Operator Manual: CAUTIC {hz_ga_F06G01U99M0	Operator Manual: CAUTIONSource: Bone CAD does not provide correct resu						
7	2025-09-29T14:38:26	TRI-e9fcbt	outside inf heart arrh	summation	Operator Manual: i Operator Man			1	Operator Manual: WARNII {hz_ga_F14G09U01M0	Operator Manual: WARNINGSource: Exceeded safety limits of total leakage						
8	2025-10-01T10:41:02	TRI-d4d4d	service ha: Damage tc	Tablet not	Operator Manual: i Operator Man			0	Operator Manual: CAUTIC {hz_ga_F13G13U05M0	Operator Manual: CAUTIONSource: Tablet not in the cradle during updates						
9	2025-10-01T10:47:46	TRI-e9fcbt	outside inf heart arrh	summation	Operator Manual: i Operator Man			1	Operator Manual: WARNII {hz_ga_F14G09U01M0	Operator Manual: WARNINGSource: Exceeded safety limits of total leakage						
10	2025-10-01T10:48:41	TRI-dcc0el	outside inf damage tc	fire aroun	Operator Manual: i Operator Man			0	Operator Manual: WARNII {hz_ga_F14G03U01M0	Operator Manual: WARNINGSource: Fire inside or in the vicinity of the syst						
11	2025-10-01T10:52:35	TRI-bde8b	post-procc wrong dia	Wrong dia	Operator Manual: i Operator Man			0	Operator Manual: CAUTIC {hz_ga_F06G01U118M	Operator Manual: CAUTIONSource: Automatically generated results, archive						
12	2025-10-01T10:53:00	TRI-1d233	archiving	Data lost	Low-quali	Operator Manual: CAUTIONSourc		0	Operator Manual: CAUTIC {hz_ga_F08G02U06M0	Operator Manual: CAUTIONSource: Low-quality storage media are used						
13	2025-10-01T11:03:44	TRI-1d233	archiving	Data lost	Low-quali	Operator Manual: CAUTIONSourc		0	Operator Manual: CAUTIC {hz_ga_F08G02U06M0	Operator Manual: CAUTIONSource: Low-quality storage media are used						
14	2025-10-02T14:19:49	TRI-65b78	Scanner M (ICU) Syste	Damage tc	Operator Manual: i Operator Man			0	Operator Manual: WARNII {hz_ga_F25G01U14M0	Operator Manual: WARNINGSource: The system is not properly prepared to						
15	2025-10-03T16:50:35	TRI-e3ee1	patient reg incorrect c	wrong ent	Operator Manual: i Operator Man			0	Operator Manual: CAUTIC {hz_ga_F01G01U03M0	Operator Manual: CAUTIONSource: Wrong entry of patient position						
16	2025-10-20T20:06:12	TRI-1d233	archiving	Data lost	Low-quali	Operator Manual: CAUTIONSourc		0	Operator Manual: CAUTIC {hz_ga_F08G02U06M0	Operator Manual: CAUTIONSource: Low-quality storage media are used						
17	2025-10-20T20:13:19	TRI-74d4d	Scanner M Damage tc	(ICU) Wire	Operator Manual: CAUTIONSourc			0	Operator Manual: CAUTIC {hz_ga_F25G01U13M0	Operator Manual: CAUTIONSource: Improper positioning of the hydraulic fe						
18	2025-10-20T20:15:51	TRI-bde8b	post-procc wrong dia	Wrong dia	Operator Manual: i Operator Man			1	Operator Manual: CAUTIC {hz_ga_F06G01U118M	Operator Manual: CAUTIONSource: Automatically generated results, archive						

Figure A.20: Administrative Review and Archival.

A.3 User Documentation

This section explains how to operate the Safety Chatbot prototype from setup to export. The steps below describe the standard workflow used by safety engineers and administrators when running the system locally.

A.3.1 Activate the Python Virtual Environment

Open a terminal in the project directory and activate the virtual environment to load all required dependencies.

```
source .venv/bin/activate
.\.venv\Scripts\activate
```

A.3.2 Build the Index and Embeddings

Ensure the Excel dataset containing validated safety messages is located in the data/ folder. Run the indexing script to load the data, create embeddings, and store them in the local FAISS database.

```
python build_index.py
```

After execution, the terminal displays the embedding model information, vector dimensions, and the number of indexed chunks. Once indexing is complete, perform a quick sanity check by previewing the Excel entries and verifying the embedding output and *Ollama* test response to confirm that retrieval and generation are functioning correctly.

A.3.3 Launch the Streamlit Application

Start the user interface by running the Streamlit script.

```
streamlit run matchref_chat_app.py
```

This command opens the chatbot interface on <http://localhost:8501>.

A.3.4 Enter Input Data

The interface sidebar provides instructions for entering data in the required FHC format. Each field must be filled out exactly as specified:

```
function=Patient Registration  
hazard =Incorrect Diagnosis  
cause=Wrong Entry of Patient Orientation
```

If the format or structure is incorrect, the chatbot requests the user to recheck the input before proceeding.

A.3.5 Retrieve or Generate Safety Message

After input validation, the system searches the FAISS database:

- If an exact match is found, one or two validated safety messages are displayed.
- If no exact match exists, the local LLM (*Ollama*) generates a new message following a strict JSON structure.

A.3.6 Review, Edit, and Confirm

Review the suggested message. If required, make small edits directly in the interface to ensure clarity and compliance. When satisfied, click Confirm. The system then assigns or retrieves the corresponding Hazard Key (HRK) and records the action in the validation log.

A.3.7 Export the Results

Type export into the chatbot input field to save all data. This creates or updates two Excel files in the data/ directory:

- validation_log.xlsx – complete record of inputs, outputs, and confirmations.
- hazard_requirement_key_list.xlsx – list of all HRKs with associated safety messages.

A.3.8 Completion and Administration

A session is complete once the validated message is confirmed and both Excel files are successfully updated. Administrators should review and archive these logs periodically to maintain data integrity. Future enterprise integration will include role based access control and automated log management.

Bibliography

- [1] American National Standards Institute. *ANSI_Z535.6_2023_Analysis [Excel file]*. `path/to/ANSI_Z535.6_2023_Analysis.xlsx`. Unpublished internal analysis. 2023.
- [2] C. Baur, S. Denner, B. Wiestler, N. Navab, and S. Albarqouni. “Autoencoders for unsupervised anomaly segmentation in brain MR images: A comparative study”. In: *Medical Image Analysis* (2021).
- [3] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, et al. “On the Opportunities and Risks of Foundation Models”. In: *arXiv preprint arXiv:2108.07258* (2021).
- [4] T. B. Brown et al. *GPT-3: Language Models are Few-Shot Learners*. OpenAI Blog. 2020.
- [5] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 1877–1901.
- [6] N. Catalyst. “What Is Risk Management in Healthcare?” In: *NEJM Catalyst* (2018).
- [7] I. E. Commission. *IEC 62366-1:2015+AMD1:2020 CSV – Medical devices – Part 1: Application of usability engineering to medical devices / Dispositifs médicaux – Partie 1: Application de l’ingénierie de l’aptitude à l’utilisation aux dispositifs médicaux*. standard. Consolidated version in English and French (en-fr). 2020.
- [8] A. De Gregorio et al. “Risk Governance and Transparency in High-Risk AI Systems”. In: *AI Ethics Review* 2.1 (2023), pp. 15–29.

- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *NAACL-HLT* (2019).
- [10] S. Diemert and J. H. Weber. “Can Large Language Models Assist in Hazard Analysis?” In: *Proceedings of the International Conference on Software Engineering in Health Care (SEHC)*. 2023.
- [11] European Commission. *Regulation (EU) 2024/1689 of the European Parliament and of the Council laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. <https://commission.europa.eu/ai-act/>. Official text of the EU AI Act. 2024.
- [12] European Commission High-Level Expert Group on AI. *Ethics Guidelines for Trustworthy AI*. European Commission, 2019.
- [13] European Parliament and Council. *Regulation (EU) 2017/745 of the European Parliament and of the Council of 5 April 2017 on medical devices, amending Directive 2001/83/EC, Regulation (EC) No 178/2002 and Regulation (EC) No 1223/2009 and repealing Council Directives 90/385/EEC and 93/42/EEC*. Accessed: 2025-08-15. 2017.
- [14] A. Fatema et al. “Differential Privacy and Federated Learning for Clinical AI”. In: *Frontiers in Healthcare Technology* 4.1 (2023), pp. 11–23.
- [15] E. Ferrara. “Fairness and Bias in Artificial Intelligence: A Brief Survey of Sources, Impacts, and Mitigation Strategies”. In: *arXiv preprint arXiv:2401.00000* (2024).
- [16] L. Floridi. “Establishing the Rules for Building Trustworthy AI”. In: *Nature Machine Intelligence* 1.6 (2019), pp. 261–262.
- [17] A. Gatt and E. Krahmer. “Natural Language Generation”. In: *The Oxford Handbook of Computational Linguistics*. Ed. by R. Mitkov. Oxford University Press, 2018, pp. 336–360.
- [18] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. “Generative Adversarial Nets”. In: *Advances in Neural Information Processing Systems* 27 (2014).

- [19] Greenlight Guru. *ISO 14971 Risk Management for Medical Devices: The Definitive Guide*. Accessed: November 2025. 2023. URL: <https://www.greenlight.guru/blog/iso-14971-risk-management>.
- [20] M. C. Horowitz, G. Allen, E. B. Kania, and P. Scharre. “A Unified Framework of Five Principles for AI in Society”. In: *Harvard Data Science Review* 1.1 (2019).
- [21] G. Hounsfield and A. Cormack. *Computed Tomography: Development and Application*. Medical Imaging Conference Proceedings, 1973.
- [22] International Electrotechnical Commission. “IEC 62366-1: Application of Usability Engineering to Medical Devices”. In: *International Standard* (2015).
- [23] International Electrotechnical Commission and Institute of Electrical and Electronics Engineers. *IEC/IEEE 82079-1:2019 — Preparation of information for use (instructions for use) of products — Part 1: Principles and general requirements*. Joint International Standard. 2019.
- [24] International Organization for Standardization. *Medical Devices — Application of Risk Management to Medical Devices*. International Standard ISO 14971:2019. Geneva, Switzerland: International Organization for Standardization, 2019.
- [25] International Organization for Standardization. *ISO/IEC 42001:2023 — Artificial Intelligence Management System*. <https://www.iso.org/standard/81230.html>. International Standard for AI Management Systems. 2023.
- [26] Intuition Labs. “AI and LLMs in Regulatory Affairs: Automating Documentation for Compliance”. In: (2024). Accessed: November 2025.
- [27] G. Izacard and E. Grave. “Unsupervised Dense Information Retrieval with Contrastive Learning”. In: *arXiv preprint arXiv:2104.07067* (2021).
- [28] John Snow Labs. “The Power of Medical Large Language Models (LLMs) in Healthcare”. In: (2024). Accessed: November 2025.
- [29] J. Johnson, M. Douze, and H. Jégou. “Billion-scale similarity search with FAISS”. In: *IEEE Transactions on Big Data*. IEEE, 2019.
- [30] D. Jurafsky and J. H. Martin. *Speech and Language Processing*. Pearson, 2023.

- [31] O. Khattab and M. Zaharia. “ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT”. In: *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2020, pp. 39–48.
- [32] D. P. Kingma and M. Welling. “Auto-Encoding Variational Bayes”. In: *arXiv preprint arXiv:1312.6114* (2013).
- [33] A. Laitenberger. *Expanding Horizons in RAG: Exploring and Extending the Limits of RAPTOR*. Stanford CS224N Custom Project. 2024.
- [34] P. Lee, J. Adams, N. Patel, et al. “Large Language Models for Reducing Clinician Workload in Healthcare Communication”. In: *The Lancet Digital Health* (2024). Accessed: November 2025.
- [35] P. Lewis, E. Perez, A. Piktus, et al. “Retrieval-augmented generation for knowledge-intensive NLP tasks”. In: *Advances in Neural Information Processing Systems*. 2020.
- [36] B. Li, P. Qi, B. Liu, S. Di, J. Liu, J. Pei, J. Yi, and B. Zhou. “Trustworthy AI: From Principles to Practices”. In: *arXiv preprint arXiv:2109.08207* (2021).
- [37] M. Mäntymäki, J. Salminen, and A. K. M. N. Islam. “The Hourglass Model of AI Governance: Navigating Between Ethics and Regulation”. In: *Journal of Business Ethics* 180.3 (2022), pp. 707–728.
- [38] D. J. Mayhew. “Key Principles for User-Centred Systems Design”. In: *Behaviour & Information Technology* 22.6 (2003), pp. 397–409.
- [39] S. Min, P. Lewis, L. Wu, D. Hall, et al. “A Survey of Retrieval-Augmented Generation”. In: *Transactions of the Association for Computational Linguistics* (2023).
- [40] U. Muehlematter et al. “AI Governance in Medical Device Software”. In: *HealthTech Journal* 19.3 (2021), pp. 45–60.
- [41] H. Nori, N. King, D. Carignan, et al. “Capabilities of Clinical Large Language Models in Biomedical Text Understanding”. In: *npj Precision Oncology* 9.1 (2025), pp. 1–12.

- [42] D. A. Norman and S. W. Draper, eds. *User Centered System Design: New Perspectives on Human-Computer Interaction*. Hillsdale, NJ: Lawrence Erlbaum Associates, 1986.
- [43] J. F. Nunamaker, M.-T. Chen, and T. D. M. Purdin. “Systems development in information systems research”. In: *Journal of Management Information Systems* 7.3 (1990), pp. 89–106.
- [44] OpenAI et al. “GPT-4 Technical Report”. In: *arXiv preprint arXiv:2303.08774* (2023).
- [45] E. Papagiannidis, P. Mikalef, and K. Conboy. “Responsible Artificial Intelligence Governance: A Review and Research Framework”. In: *Information Systems Frontiers* (2023).
- [46] T. Pieters and L. Landeweerd. “A Structured Approach to Risk Management in Medical Devices: Ensuring Safety through Identification, Documentation, and Mitigation”. In: *Biomedical Engineering Online* 20.1 (2021), pp. 1–15.
- [47] A. Radford et al. *Language models are unsupervised multitask learners*. OpenAI Blog. 2019.
- [48] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. *Improving language understanding by generative pre-training*. OpenAI Blog. 2018.
- [49] *Regulation (EU) 2016/679 of the European Parliament and of the Council. General Data Protection Regulation (GDPR)*. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>. Official Journal of the European Union. 2016.
- [50] *Regulation (EU) 2024/XXX of the European Parliament and of the Council. Artificial Intelligence Act (AI Act)*. <https://data.consilium.europa.eu/doc/document/PE-00-0000-2024/init/en/pdf>. Official Journal of the European Union, final adopted text. 2024.
- [51] D. Ribeiro, T. Rocha, G. Pinto, B. Cartaxo, M. Amaral, N. Davila, and A. Camargo. “Toward Effective AI Governance: A Review of Principles”. In: *arXiv preprint arXiv:2303.XXXXX* (2023).

- [52] A. Rogers, O. Kovaleva, and A. Rumshisky. “A Primer in BERTology: What We Know About How BERT Works”. In: *Transactions of the Association for Computational Linguistics* 8 (2020), pp. 842–866.
- [53] D. Sarthi, A. Arora, and P. Talukdar. “Recursive Abstractive Processing for Tree-Organized Retrieval (RAPTOR)”. In: *arXiv preprint arXiv:2404.11302* (2024).
- [54] R. Sennrich, B. Haddow, and A. Birch. “Neural Machine Translation of Rare Words with Subword Units”. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2016.
- [55] T. Sera. “Computed Tomography”. In: *Transparency in Biology*. Ed. by K. Soga, M. Umezawa, and K. Okubo. Springer, 2021.
- [56] Y. Shao, Y. Chen, T. Zhang, et al. “MedAlign: A Clinician-Expert-Annotated Dataset for Aligning Large Language Models with Clinical Tasks”. In: *arXiv preprint* (2023). eprint: **2308.14089**.
- [57] Siemens Healthcare. *Manual – Authoring Guidelines SHS DI/AT 5*. Internal documentation.
- [58] H. Stix et al. “AI Governance and Regulation: Challenges and New Approaches”. In: *AI and Society* 36.2 (2021), pp. 345–356.
- [59] A. J. Thirunavukarasu, D. Ting, D. Elangovan, et al. “Large Language Models in Medicine”. In: *Nature Medicine* 29 (2023), pp. 1930–1940.
- [60] U.S. Congress. *Health Insurance Portability and Accountability Act of 1996*. Public Law 104–191, 110 Stat. 1936. Codified at 42 U.S.C. §1320d et seq. 1996.
- [61] U.S. Food and Drug Administration. *Human Factors Considerations for Medical Device Design and Risk Analysis: Guidance for Industry and FDA Staff*. Tech. rep. Accessed: November 2025. Center for Devices and Radiological Health (CDRH), 2023.
- [62] A. Vaswani et al. “Attention is all you need”. In: *Advances in neural information processing systems*. 2017.
- [63] P. Voigt and A. von dem Bussche. *The EU General Data Protection Regulation (GDPR): A Practical Guide*. Springer, 2017.

- [64] J. Wecklein. *Risk Management Plan, Version 64*. Siemens Healthineers, SHS DI CT QT QA RM, last updated May 15, 2024. Internal document. 2024.
- [65] J. Wei and X. Wang. “Explainable AI and Transparency Measures in Clinical Decision Support”. In: *Journal of AI in Medicine* 12.3 (2023), pp. 45–59.
- [66] C. Wharton, J. Rieman, C. Lewis, and P. Polson. “The Cognitive Walkthrough Method: A Practitioner’s Guide”. In: *Usability Inspection Methods* (1994). Ed. by J. Nielsen and R. L. Mack, pp. 105–140.
- [67] T. Young, D. Hazarika, S. Poria, and E. Cambria. “Recent trends in deep learning based natural language processing”. In: *IEEE Computational Intelligence Magazine* 13.3 (2018), pp. 55–75.

Declaration of authorship

I confirm that I have written this thesis unaided and without using sources other than those listed and that this thesis has never been submitted to another examination authority and accepted as part of an examination achievement, neither in this form nor in a similar form. All content that was taken from a third party, either verbatim or in substance, has been acknowledged as such.

Uttenrueth, 01.12.2025

Rithika Ravichandran

Acknowledgements

I would like to express my sincere gratitude to the UDOC team at Siemens Healthineers for giving me the opportunity to work on this topic as my master's thesis. My heartfelt thanks go to my academic supervisor, Professor Dr. Jana Hutter, for her continuous guidance and for helping me balance the academic and industrial requirements of this work.

I am deeply grateful to my industry supervisor, M.Sc. Stephanie Heidepreim, whose support, encouragement, and valuable insights guided me throughout the project. I would also like to thank my manager, Frank Seemann, and our domain expert, Claudia Scholz, for their generous assistance, constructive feedback, and for creating an environment in which I could learn and grow.

My heartfelt appreciation goes to my family and friends abroad for their unwavering encouragement and for motivating me to pursue and complete this master's program. I am especially thankful to my friends Kowshik, Harini, Vignesh, Sonia, Pavan, and Debaritha for their steadfast support and for being my pillars of strength during my time here.

Starting this journey in Germany was completely new to me, and I've loved every moment of it, from the learning to the friendships. I'm truly thankful to everyone who has been a part of this chapter.